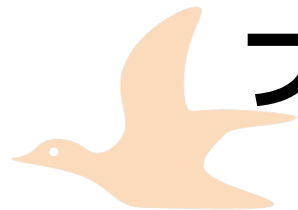
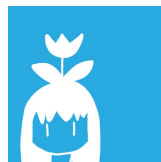


プログラミング言語処理系入門



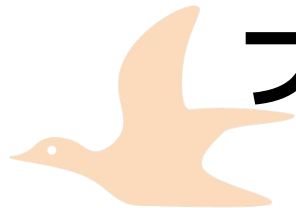
プログラミング言語を作る理論

コンピューターは四則演算の順序をいかに認識するのか



X(Twitter) : ShioPy0101

プログラミング言語処理系入門

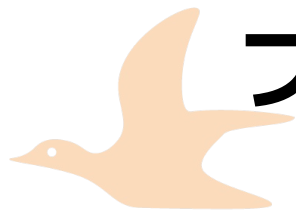


プログラミング言語を作る理論

コンピューターは四則演算の順序をいかに認識するのか

(1)

プログラミング言語処理系入門

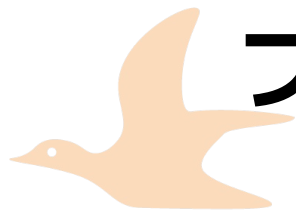


プログラミング言語を作る理論

コンピューターは四則演算の順序をいかに認識するのか

(2)

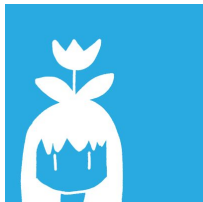
プログラミング言語処理系入門



プログラミング言語を作る理論

コンピューターは四則演算の順序をいかに認識するのか

(3)



- ・ 20才
- ・ 今回の内容はほとんど独学

趣味

- ・ 映像制作 / プログラミング

よく使う言語

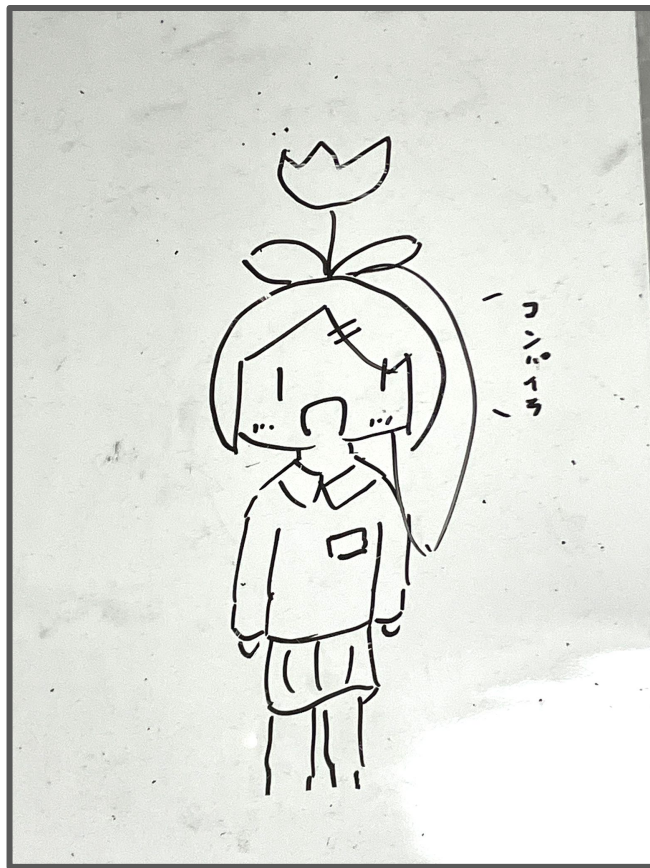
- ・ C++ / TypeScript / Python

よく使うライブラリ

- ・ React / OpenCV / Numpy

好きなもの

- ・ りんご / アイスクリーム



構文解析・文脈自由文法可視化ツール（自作）



<https://lr-learning.vercel.app/>



しおまち

@ShioPy0101

...

lr-learning.vercel.app

LR構文解析や、文脈自由文法による構文定義、shift-reduce parsing による木構造の生成過程をグラフィカルに表示するツールを開発したので、一般公開します！（勉強会で使用したものです！）

午後10:17・2025年9月9日・6,604 件の表示

📊 ポストのエンゲージメントを表示



↻ 22

❤ 73

🔖 33





第1章

プログラミング 言語処理系って？

-プログラムは本来ただの文字列-



プログラミング言語処理系って？

たいへんあやしいC++プログラム

```
9 namespace DFAParse
10 void ClosureExpansion::recursionNodeClosureExpansion(LRItemStruct &lr_item, string search_key, int LR_formula_expansion_vector_index, BNFParser::V
115 for (int k = 0; k < dexp_size(); k++)
116 if (HasKeyMap[getHasKeyString(already_explored_formula_expansion), dexp[k].formula_expansion_label])
117 int la_size = lr_item.LR_formula_map[token.token_str].LR_formula_expansion_vector[n_index].look_ahead_size();
118 for (int t = 0; t < la_size; t++)
119 {
120     BNFParser::VDeploymentTokenStruct new_first_set = getLatterFirstSet(lr_item.LR_formula_map[token.token_str].LR_formula_expansion_ve
121     recursionNodeClosureExpansion(lr_item, token.token_str, n_index, new_first_set);
122 }
123 }
124 else
125 { // 先読み記号だけ追加する
126     int formula_expansion_index = already_explored_formula_expansion[dexp[k].formula_expansion_label];
127     BNFParser::VDeploymentTokenStruct *current_look_ahead_p = &(lr_item.LR_formula_map[token.token_str].LR_formula_expansion_vector[formula
128 }
129 for (int n = 0; n < first_set.size(); n++)
130 {
131     bool flag = true; // 既に登録されているか
132     for (int s = 0; s < current_look_ahead_p->size(); s++)
133     {
134         if (current_look_ahead_p->at(s).token_str == first_set[n].token_str)
135         {
136             flag = false;
137             break;
138         }
139     }
140     if (flag)
141     {
142         current_look_ahead_p->push_back(first_set[n]);
143     }
144     int dot = lr_item.LR_formula_map[token.token_str].LR_formula_expansion_vector[formula_expansion_index].dot;
145     BNFParser::VDeploymentTokenStruct new_first_set = getLatterFirstSet(lr_item.LR_formula_map[token.token_str].LR_formula_expansio
146     recursionNodeClosureExpansion(lr_item, token.token_str, formula_expansion_index, new_first_set);
147 }
148 }
149 }
150 }
```

高水準原始プログラムは、
コンピューターは理解できない
もっとシンプルな体系にする必要



わからない
(直接解釈・実行できない)





プログラミング言語処理系って？

たいへんあやしいC++プログラム

```
9 namespace DFAParse
10 void ClosureExpansion::recursionNodeClosureExpansion(LRItemStruct &lr_item, string search_key, int LR_formula_expansion_vector_index, BNFParser::V
115 for (int k = 0; k < dexp_size(); k++)
116 if (HasKeyMap[getMapKeyString(already_explored_formula_expansion), dexp[k].formula_expansion_label])
117 int la_size = lr_item.LR_formula_map[token.token_str].LR_formula_expansion_vector[n_index].look_ahead_size();
118 for (int t = 0; t < la_size; t++)
119 {
120 BNFParser::VDeploymentTokenStruct new_first_set = getLatterFirstSet(lr_item.LR_formula_map[token.token_str].LR_formula_expansion_ve
121 recursionNodeClosureExpansion(lr_item, token.token_str, n_index, new_first_set);
122 }
123 }
124 else
125 { // 先読み記号だけ追加する
126 int formula_expansion_index = already_explored_formula_expansion[dexp[k].formula_expansion_label];
127 BNFParser::VDeploymentTokenStruct <current_look_ahead_p = &(lr_item.LR_formula_map[token.token_str].LR_formula_expansion_vector[formula
128 for (int n = 0; n < first_set.size(); n++)
129 {
130 bool flag = true; // 既に登録されているか
131 for (int s = 0; s < current_look_ahead_p->size(); s++)
132 {
133 if (current_look_ahead_p->at(s).token_str == first_set[n].token_str)
134 {
135 flag = false;
136 break;
137 }
138 }
139 if (flag)
140 {
141 current_look_ahead_p->push_back(first_set[n]);
142 }
143 int dot = lr_item.LR_formula_map[token.token_str].LR_formula_expansion_vector[formula_expansion_index].dot;
144 BNFParser::VDeploymentTokenStruct new_first_set = getLatterFirstSet(lr_item.LR_formula_map[token.token_str].LR_formula_expansio
145 recursionNodeClosureExpansion(lr_item, token.token_str, formula_expansion_index, new_first_set);
146 }
147 }
148 }
149 }
```

翻訳

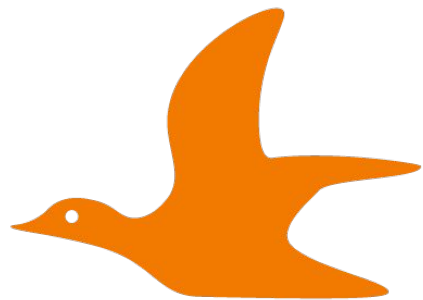
規定された
文法以外を排除



わかる！



プログラミング言語処理系って？



プログラムは本来
ただの文字列

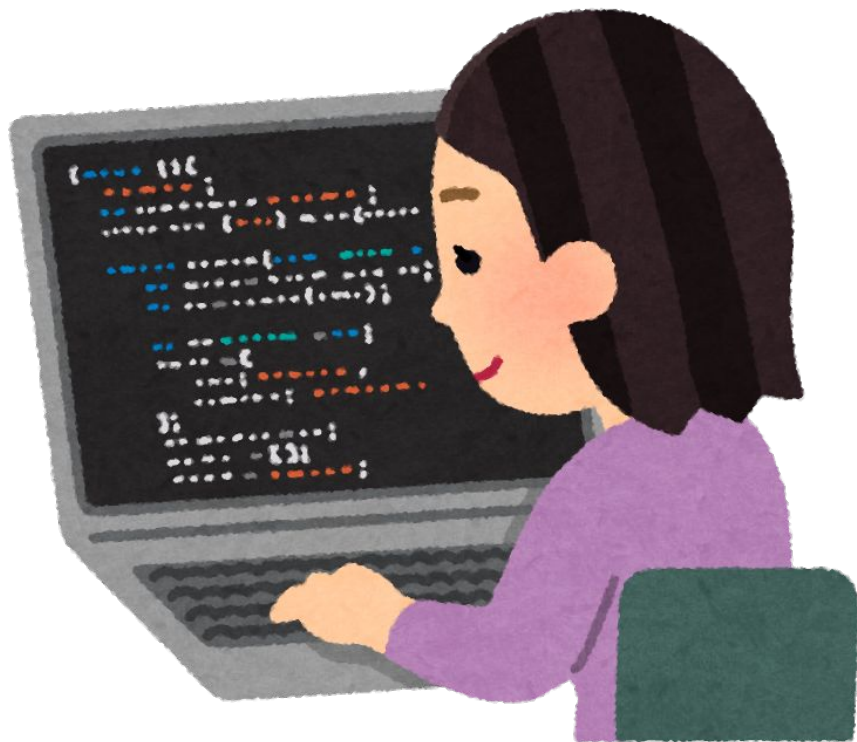


プログラミング言語処理系って？

カラフルな文字

カラフルな事は
実行には、なんの役にも立たない
あくまで、人間が理解しやすいように

最悪、メモ帳でも
プログラミングはできる (したくない)





プログラミング言語の様式

プログラミング言語はただの文字列

ただの文字列とは言っても、、、

**動く！
実行できる！**



プログラミング言語の様式

プログラミング言語はただの文字列

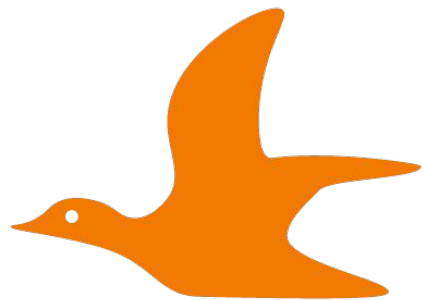


無規則な文字列では
意味がない

キーボードを叩いただけとか . . .



プログラミング言語処理系って？



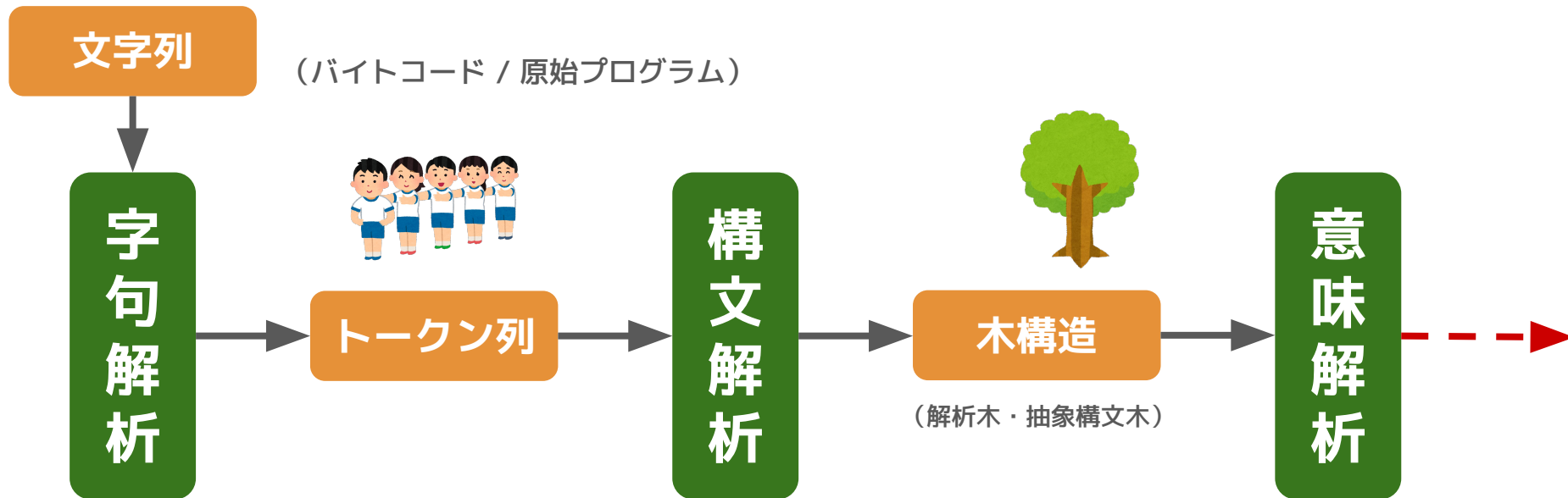
プログラムは本来
ただの文字列
(ただし、規則に縛られている)



プログラミング言語の様式

(教科書的な説明)

コンパイラの「フロントエンド」といわれる部分





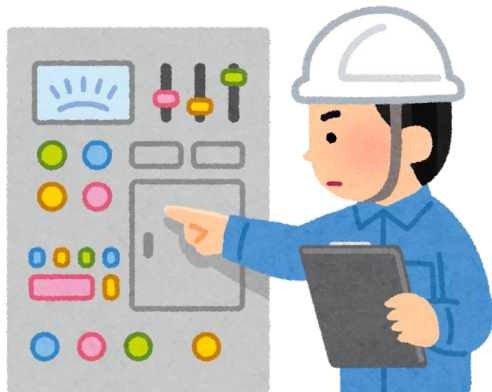
プログラミング言語の様式

(教科書的な説明)

コンパイラの「バックエンド」といわれる部分



意味解析
からの続き





プログラミング言語の様式

プログラミング言語はただの文字列

**言語そのものの
定義**

解析・実行

今回はこれを紐解いていこうということ！



プログラミング言語の様式

実装するには？

字句解析

構文解析

意味解析

最適化

生成

がんばる

がんばる

がんばる

がんばる

がんばる

「文字列をいじってるだけ」
言い返せば全部気合でできてしまう
これが面白いところ・・・（？）



プログラミング言語の様式

実装するには？

字句解析

構文解析

意味解析

最適化

生成

がんばる

がんばる

がんばる

がんばる

がんばる

「実装を気合でがんばる」でもいいのだが・・・
実装が非効率的になったり、速度ができなかったりする

確立されたアルゴリズムを使おう！



第2章

字句解析編

- 実は簡単 -



字句解析

○○解析・・・ってつくと
難しそうですが、
(結構例外的に)
実は、字句解析は簡単



字句解析

```
const hoge = 256;
```

のようなプログラムが字句解析器に渡されたとき

const

hoge

=

256

;

のように分割し、

ConstKeyword

Identifier

First
Assignment

FirstLiteral
Token

Semicolon
Token

構文解析をサポート

のようにラベル付けするだけ・・・



字句解析

```
const hoge = 256;
```



アルファベット/数字が続く限り、参照する文字列のインデックスを進める
それ以外の文字が出現したら**トークンとして切断**

定義に従いラベルを付与

・・・を繰り返すだけでも字句解析として成り立つ

もちろん、正則表現に関する知識と、オートマトンを用いてきちんとした
理論で実装することも可能



字句解析

**TypeScript字句解析器は
ts.createScannerから使用可能**

→ラベルをts由来のものにできる



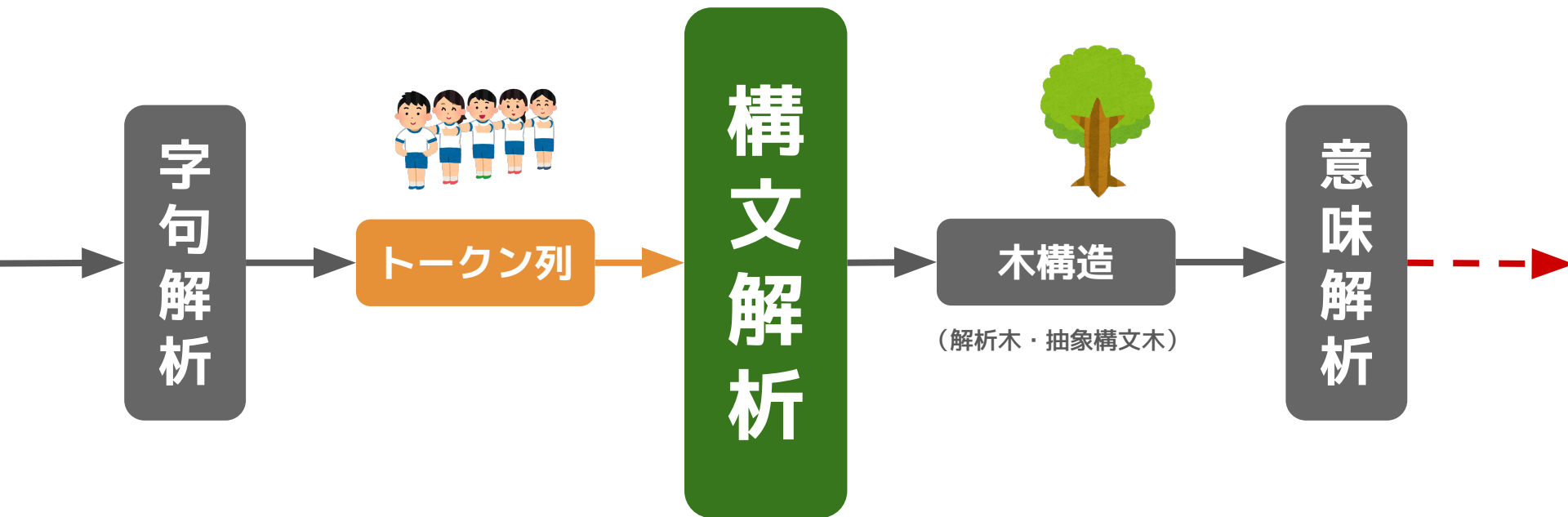
第3章

構文解析編

-木構造は津々浦々-



構文解析





構文解析

文脈自由文法
構文定義

変換

決定性有限オートマトン

変換

構文解析表
(状態遷移表)

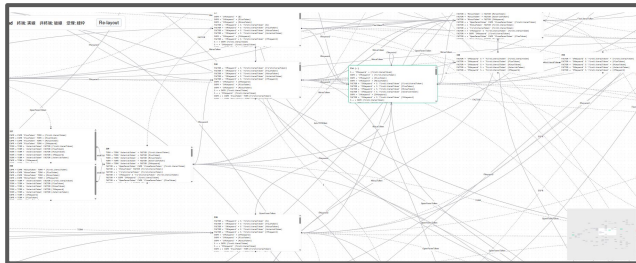
参照

トークン列

スタックを用いた
Shift-Reduce Parsing

構文木

LR(1)アイテム集合の例





構文解析



四則演算の法則

手計算してください



構文解析

こっちが先だ！



$$2 + 3 * 4$$





構文解析

$$2 + 3 * 4$$

掛け算 / 割り算が先
かっこ . . .



構文解析



四則演算の法則

**コンピューター上で
実装してください**



優先順位の設定
プログラミング言語の
構文全般 に適用できる？
if else function for . . .



構文解析

```
if (a){ . . . }  
else if (b){  
    if (b1){ . . . }  
    else  
    (b2){ . . . }  
}  
else if (c){ . . . }  
else { . . . }
```

こんなif文

正しく解析・構文定義できる？



構文解析

そこで出てくるのが
文脈自由文法 での構文定義



文脈自由文法

単純に書くと

文脈自由文法の定義

終端記号の有限集合

開始記号 $S \in N$

$$G = (N, T, P, S)$$

非終端記号の有限集合

生成規則の有限集合

$P: A \rightarrow \alpha$ の形 $A \in N, \alpha \in (N \cup \Sigma)^*$

Tは Σ として表現されることのほうが多いかも

* これ自体は授業で出てくる



文脈自由文法（終端記号 / 非終端記号）

終端記号とは

これ以上置換できない

字句解析によって作られたトークンと 直接対応するもの

字句解析

入力プログラム

3 * 4 + 5

字句解析トークン一覧（5個）：

3 FirstLiteralToken	* AsteriskToken	4 FirstLiteralToken	+ PlusToken	5 FirstLiteralToken
------------------------	--------------------	------------------------	----------------	------------------------

定義にある通り、右辺のみ使用可能

S -> EXPR

EXPR -> EXPR PlusToken TERM | EXPR MinusToken TERM | TERM

TERM -> TERM AsteriskToken FACTOR | FACTOR

FACTOR -> OpenParenToken EXPR CloseParenToken
| MinusToken FACTOR | FirstLiteralToken



文脈自由文法（終端記号 / 非終端記号）

非終端記号とは

ほかの非終端記号や、終端記号に置換できる

終端記号を用いて、関係性を記述するもの

その性質は、変数でもある

S -> EXPR

EXPR -> EXPR PlusToken TERM | EXPR MinusToken TERM | TERM

TERM -> TERM AsteriskToken FACTOR | FACTOR

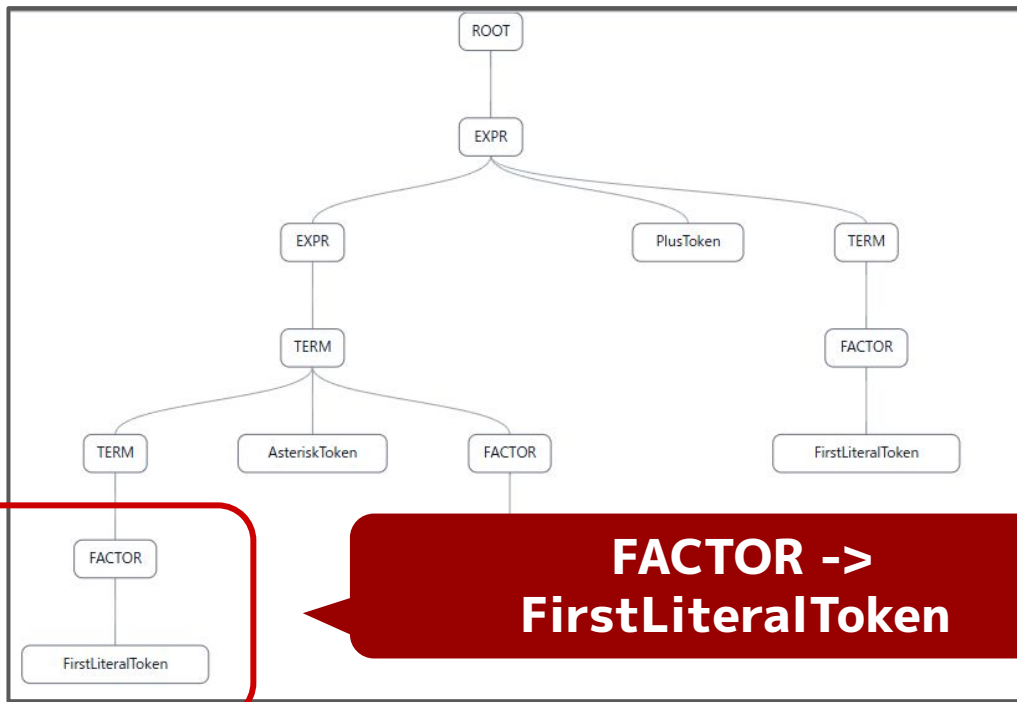
FACTOR -> OpenParenToken EXPR CloseParenToken
| MinusToken FACTOR | FirstLiteralToken



文脈自由文法（終端記号 / 非終端記号）

非終端記号とは

ほかの非終端記号や、終端記号に置換できる



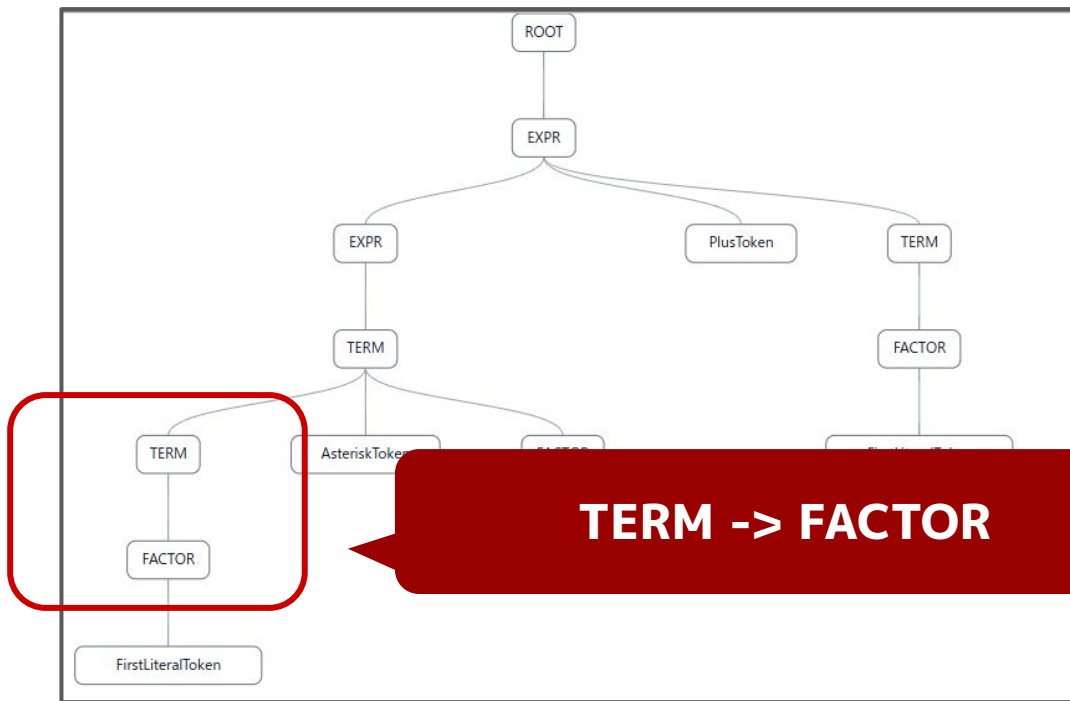
**FACTOR ->
FirstLiteralToken**



文脈自由文法（終端記号 / 非終端記号）

非終端記号とは

ほかの非終端記号や、終端記号に置換できる

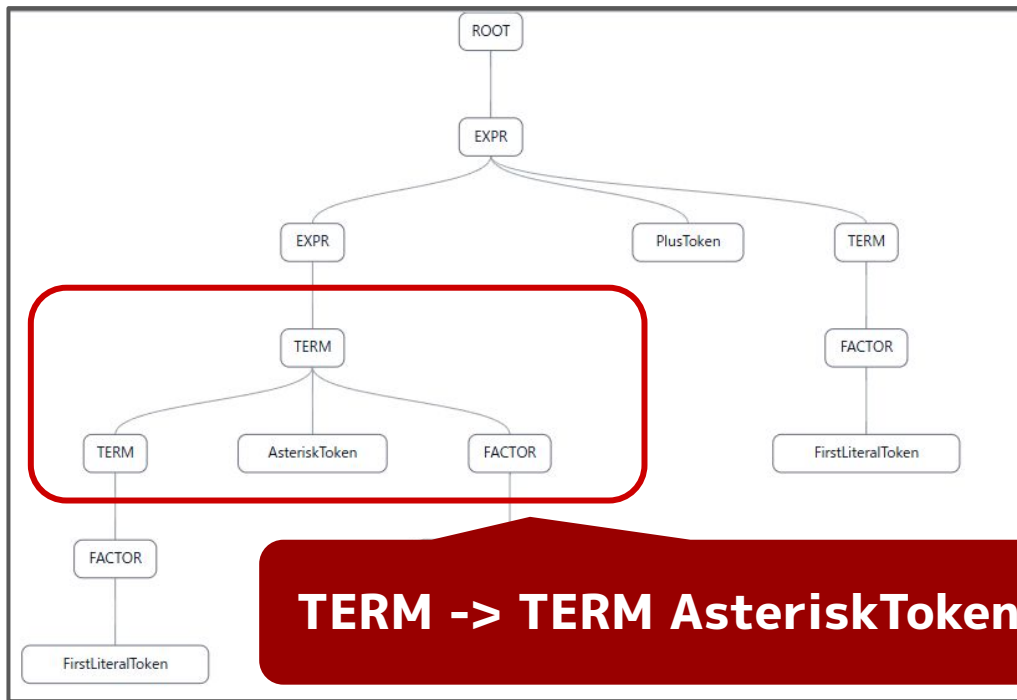




文脈自由文法（終端記号 / 非終端記号）

非終端記号とは

ほかの非終端記号や、終端記号に置換できる



TERM $\rightarrow$ TERM AsteriskToken FACTOR



文脈自由文法

単純に書くと

文脈自由文法の定義

$$G = (N, T, P, S)$$

無限通りの入力を有限の記述で指定できる！

最大の利点とも言えるかも



文脈自由文法

「無限通りの入力を有限の記述で指定できる」とは？

四則演算の構文定義

$S \rightarrow \text{EXPR}$

$\text{EXPR} \rightarrow \text{EXPR PlusToken TERM} \mid \text{EXPR MinusToken TERM} \mid \text{TERM}$

$\text{TERM} \rightarrow \text{TERM AsteriskToken FACTOR} \mid \text{FACTOR}$

$\text{FACTOR} \rightarrow \text{OpenParenToken EXPR CloseParenToken}$
 $\mid \text{MinusToken FACTOR} \mid \text{FirstLiteralToken}$

$2 + 3$

$2 + 3 * 5$

$2 + 3 * 5 + 7$

$2 + 3 * 5 + 7 * 9$

無限パターンある

これを上記の構文規則で一手に定義できている





文脈自由文法

終端記号の有限集合

開始記号 $S \in N$

$$G = (N, T, P, S)$$

非終端記号の有限集合

生成規則の有限集合
 $P: A \rightarrow \alpha$ の形 $A \in N, \alpha \in (N \cup T)^*$

- ・ $A \in N$ (A は集合 N の要素)
- ・ N は非終端記号の有限集合
- ・ 生成規則の有限集合は $A \rightarrow \alpha$ で作られる
→ よって左辺は非終端記号のみが使われる



文脈自由文法

終端記号の有限集合

開始記号 $S \in N$

$$G = (N, T, P, S)$$

非終端記号の有限集合

生成規則の有限集合

$P: A \rightarrow \alpha$ の形 $A \in N, \alpha \in (N \cup T)^*$

- ・ $\alpha \in (N \cup T)^*$
- ・ $N \cup T$ は N と T の集合をがっちゃんこしたものの（和集合）
- ・ それにアスタリスクがかかっている（クリーネ閉包）
- ・ 非終端記号の有限集合と終端記号の有限集合を合わせたものを、任意の回数(0回以上) 繰り返すことができる



文脈自由文法

単純に書くと

文脈自由文法の定義

$$G = (N, T, P, S)$$

これによって生成される言語を

文脈自由言語 という . . .



文脈自由文法

文脈自由文法の定義

$$G = (N, T, P, S)$$

文脈自由言語の定義

$$L(G) = \{ u \in T^* \mid S \Rightarrow^* u \}$$

終端記号だけからなる列
(トークン列)

開始記号 S から、有限回の導出を経て
 u
になることができる

文法 G が生成できるすべての終端記号列の集合
つまり、その文法が受け入れる文字列・原始プログラムの全集合



使用する記号たち

T^*

終端記号(Terminal Symbol)の有限集合をを T とするとき
終端記号列の全体を表す (T のクリーネ閉包)

$(T \cup N)^*$

非終端記号(Nonterminal Symbol)の有限集合を N とするとき、
終端記号と非終端記号からなる列の全体を表す

英子文字

$a, b \dots$

$\in T$ (終端記号を表す)

英子文字

$u, v \dots$

$\in T^*$ (終端記号列を表す)

字句解析の結果と思っても差支えはない

英大文字

$X, Y \dots$

$\in N$ (非終端記号を表す)

ギリシャ文字

$\alpha, \beta, \gamma \dots$

$\in (T \cup N)^*$ (任意の記号列を表す / フォントを変えています)

ただし、 ε は空文字を表す

そのほか、文字列の終端として $\$$



使用する記号たち

このような文法があるなら

$\text{EXPR} \rightarrow \text{EXPR PlusToken TERM} \mid \text{TERM}$

$\text{TERM} \rightarrow \text{NumberLiteral}$

$\text{PlusToken} \in T$

$\text{NumberLiteral} \in T$

$\text{TERM} \in N$

$\text{EXPR PlusToken TERM} \in (T \cup N)^*$

$\text{NumberLiteral PlusToken NumberLiteral} \in T^*$

$\text{NumberLiteral PlusToken NumberLiteral}$ も一応 $(T \cup N)^*$ ではある



使用する記号たち

構文規則

ならば

$$X \rightarrow \beta$$

この場合、非終端記号 X を記号列 β に置き換えることを表す

(選択演算子)

$$X \rightarrow \beta \mid \gamma$$

この場合、非終端記号 X を記号列 β もしくは γ に置き換えることを表す

構文規則の定義として、

$$X \rightarrow \beta$$

$X \rightarrow \gamma$ と独立して定義している場合と同じ



使用する記号たち

$$\alpha X \gamma \Rightarrow \alpha \beta \gamma$$

$X \rightarrow \beta$ は $\alpha X \gamma$ を $\alpha \beta \gamma$ に置き換えると表す

また、記号列 $\alpha_1, \alpha_2 \cdots \alpha_k$ ($k \geq 1$) が

$\alpha_1 \Rightarrow \alpha_2, \alpha_2 \Rightarrow \alpha_3 \cdots \alpha_{(k-1)} \Rightarrow \alpha_k$ という関係を満たすとき

$\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_k$ と表す また、導出列と呼ぶ
また、これを省略して

$\alpha_1 \Rightarrow^* \alpha_k$ とも表す

つまり、0回以上の書き換え操作によって

α_1 から α_k が導出されるとも言う



正則表現 (正規表現とも呼ぶ)

無限通りの入力を有限の記述で指定するため、以下の表記法を用いる
(1/3)

$$L(a) = \{a\}$$

正則表現 a は、文字列 a だけを含む言語を表す

$$L(r_1 \mid r_2) = L(r_1) \cup L(r_2)$$

r_1, r_2 を正則表現とすると、 $r_1 \mid r_2$ も正則表現

$$L(r_1 r_2) = L(r_1) L(r_2)$$

r_1, r_2 を正則表現とすると、接続する $r_1 r_2$ も正則表現
接続演算子を用いて表現することもある

$$\text{ただし、} L_1 L_2 = \{ xy \mid x \in L_1, y \in L_2 \}$$



正則表現 (正規表現とも呼ぶ)

無限通りの入力を有限の記述で指定するため、以下の表記法を用いる

(2/3)

このスライドではあんまり使わないけど . . .

$$L("") = \{\epsilon\}$$

このとき、二重引用符""は空列 ϵ を表す正則表現
たとえば、 $(\alpha \beta) | \epsilon$ は、言語{ "", "ab" }を表す

$$L((r)) = L(r)$$

このとき、正則表現 r をカッコで囲んだ (r) も正則表現 結合順序を変更するため使用



正則表現 (正規表現とも呼ぶ) クリーネ閉包(Kleene Closure)

無限通りの入力を有限の記述で指定するため、以下の表記法を用いる

(3/3)

$$L(r^*) = L^*(r)$$

rを正則表現とすると、r*も正則表現である
ただし、L*はLのクリーネ閉包を表す

クリーネ閉包

$$\begin{aligned} L^* &= \bigcup_{i=0}^{\infty} L^i = L^0 \cup L^1 \cup L^2 \cup L^3 \cup \dots \\ &= \{\varepsilon\} \cup L \cup LL \cup LLL \cup \dots \end{aligned}$$



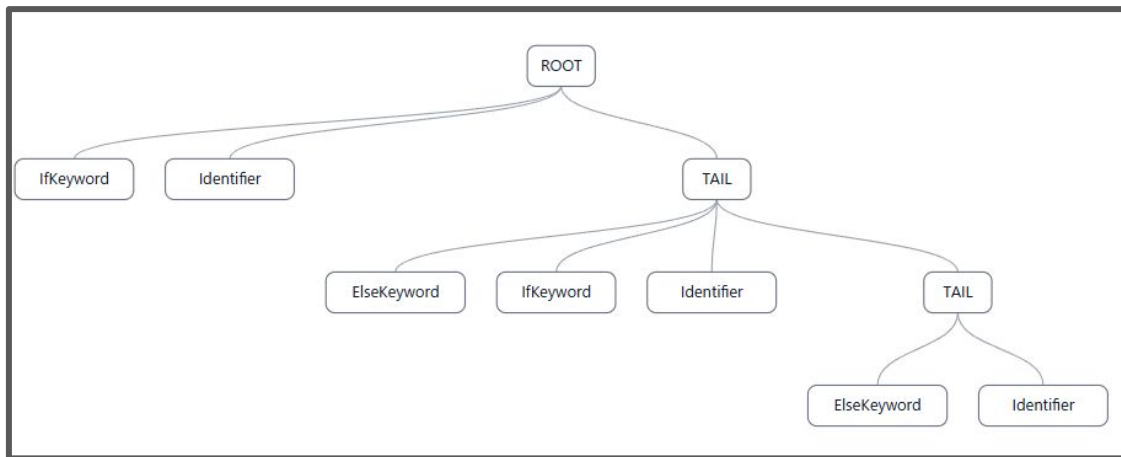
文脈自由文法

if文もどきを実現する構文規則

$S \rightarrow \text{IfKeyword Identifier TAIL}$

$\text{TAIL} \rightarrow \text{ElseKeyword IfKeyword Identifier TAIL}$
 $\quad | \text{ElseKeyword Identifier}$
 $\quad | \epsilon$

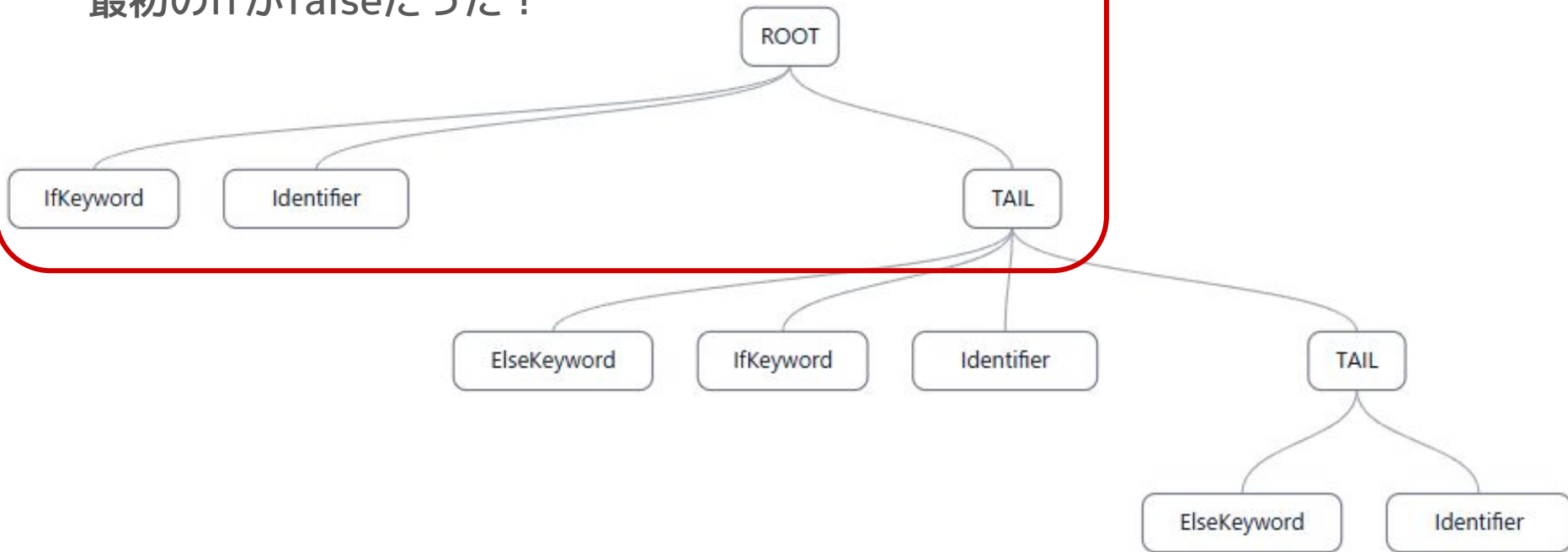
if a else if b else c
を入力





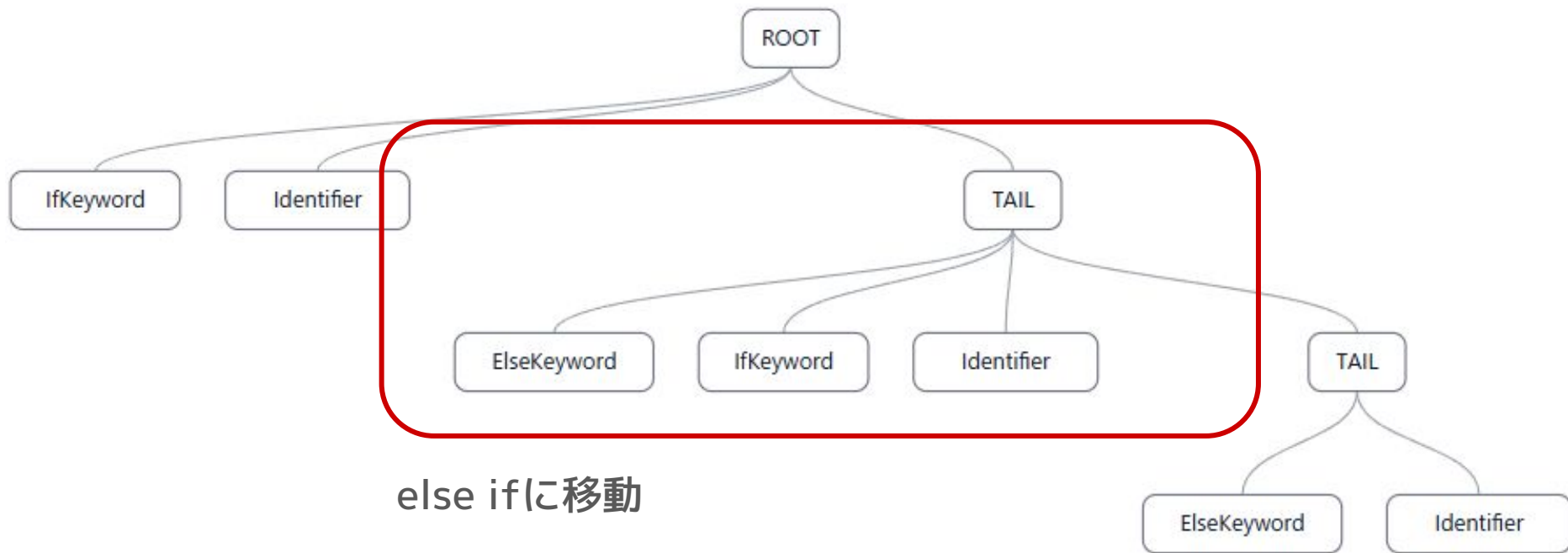
文脈自由文法

最初のifがfalseだった！



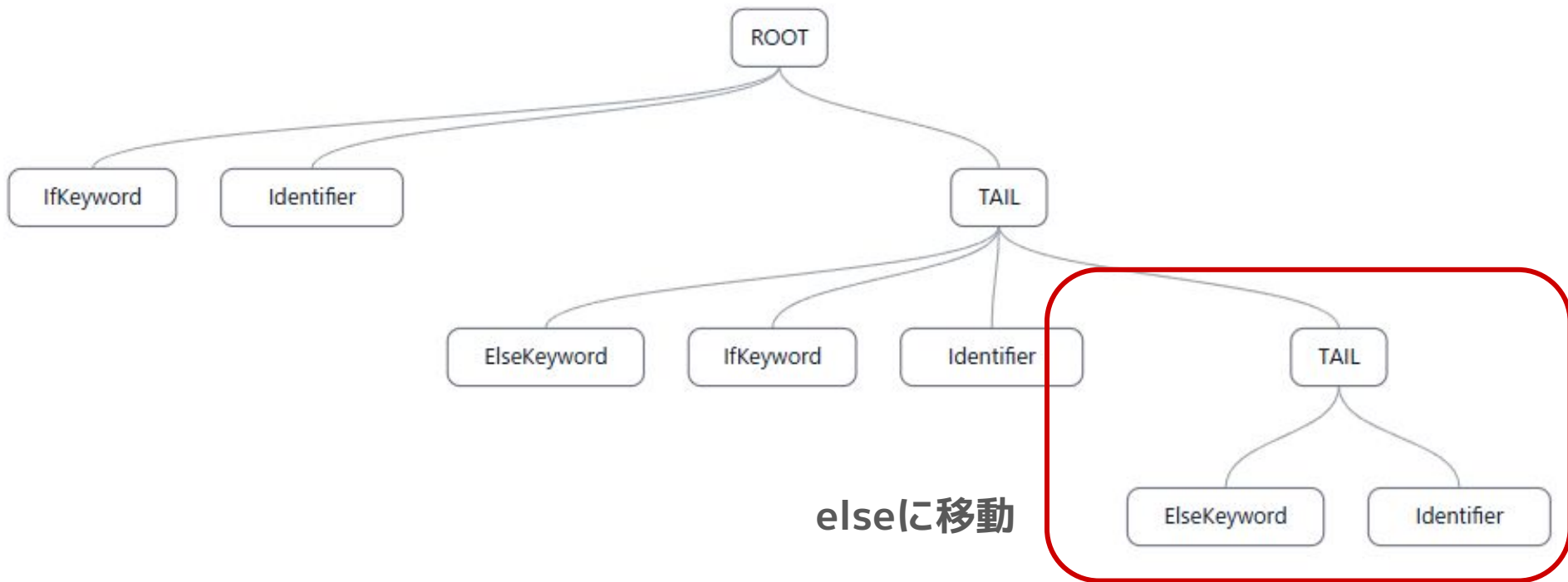


文脈自由文法





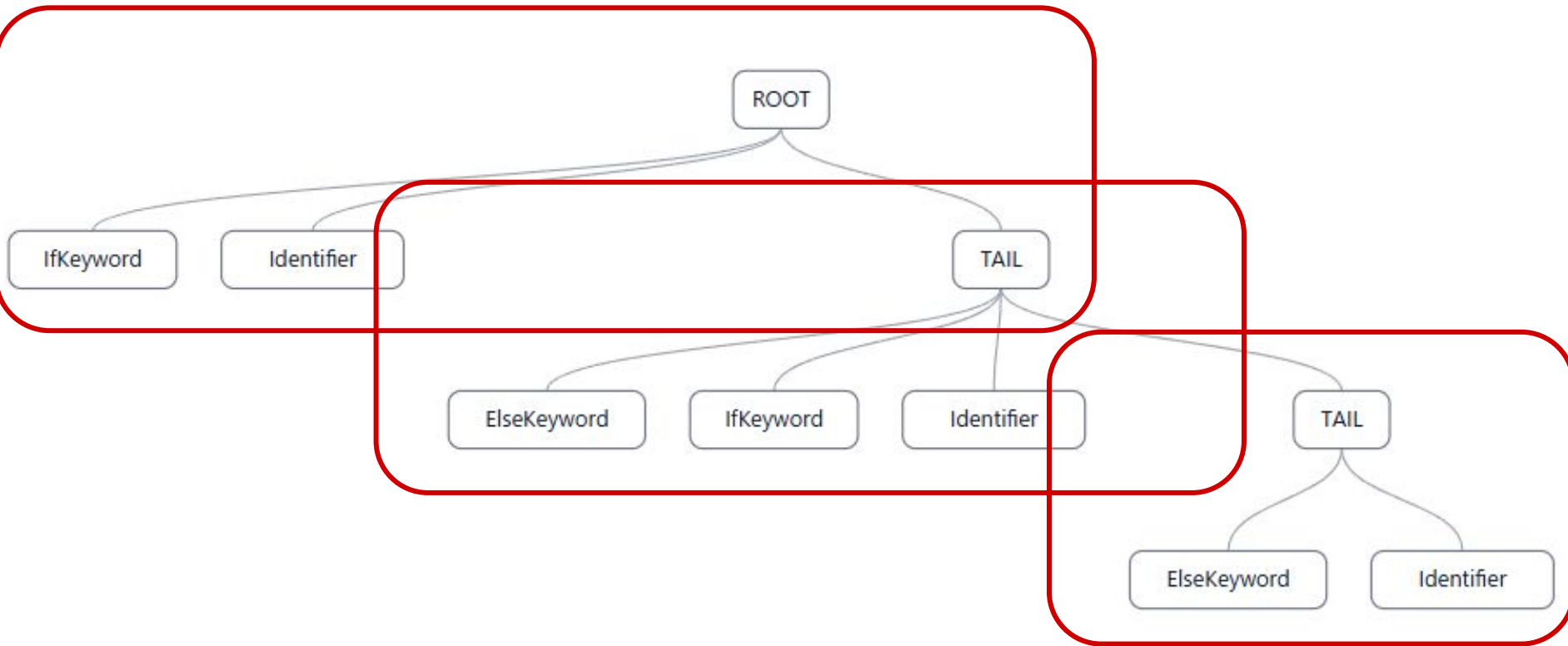
文脈自由文法





文脈自由文法

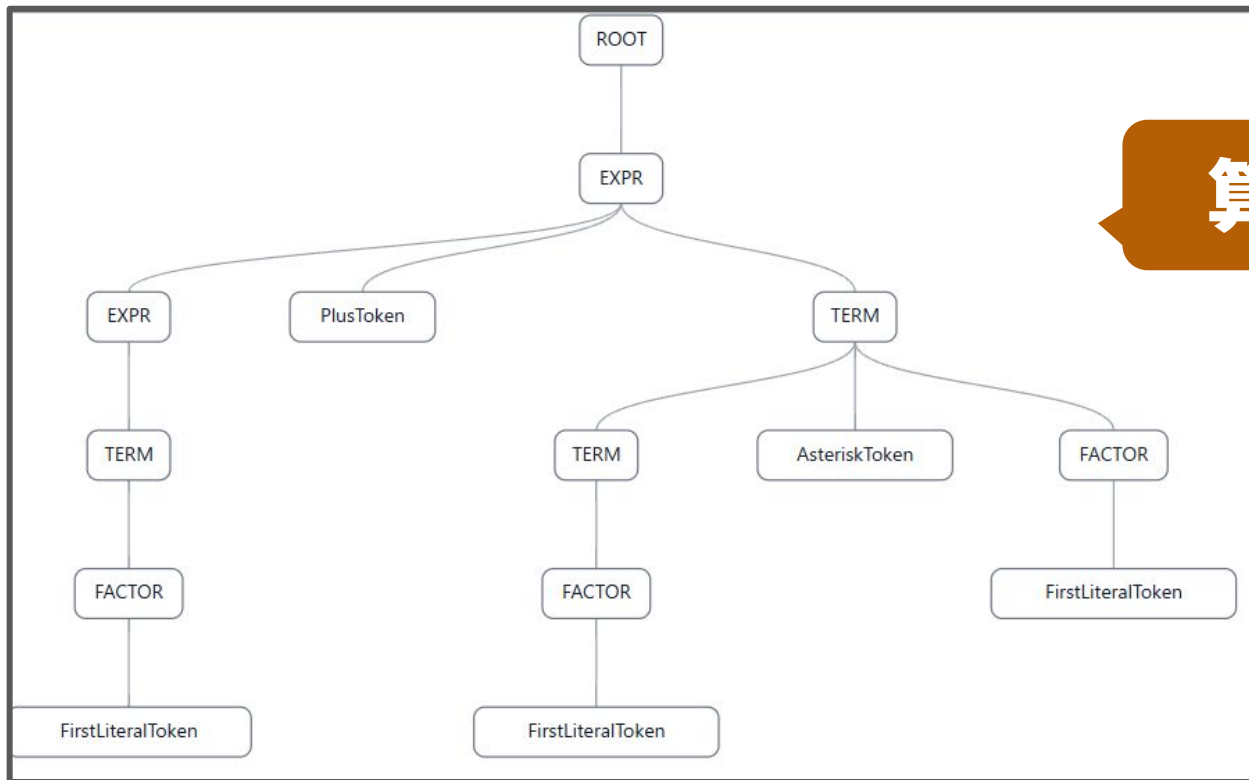
解析順序を構文定義に自然と
落とし込める！





文脈自由文法

解析順序を構文定義に自然と
落とし込める！



算術式



文脈自由文法（終端記号 / 非終端記号）

（算術式の例を含め）文脈自由文法の構文規則の定義において
非終端記号を使わずに、**任意の長さの式は定義できない**

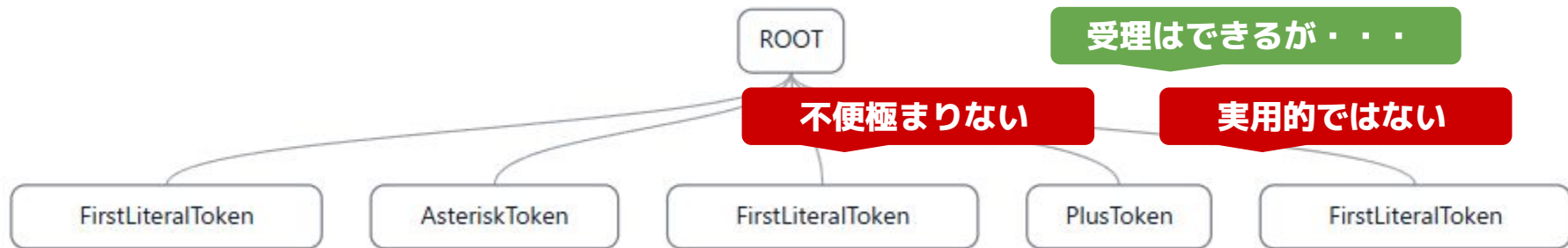
このようなトークン列を受け取りたいとき

3	*	4	+	5
FirstLiteralToken	AsteriskToken	FirstLiteralToken	PlusToken	FirstLiteralToken

構文定義（BNF）

例として）終端記号のみで構成された構文定義を作成

`S -> FirstLiteralToken AsteriskToken FirstLiteralToken PlusToken FirstLiteralToken`





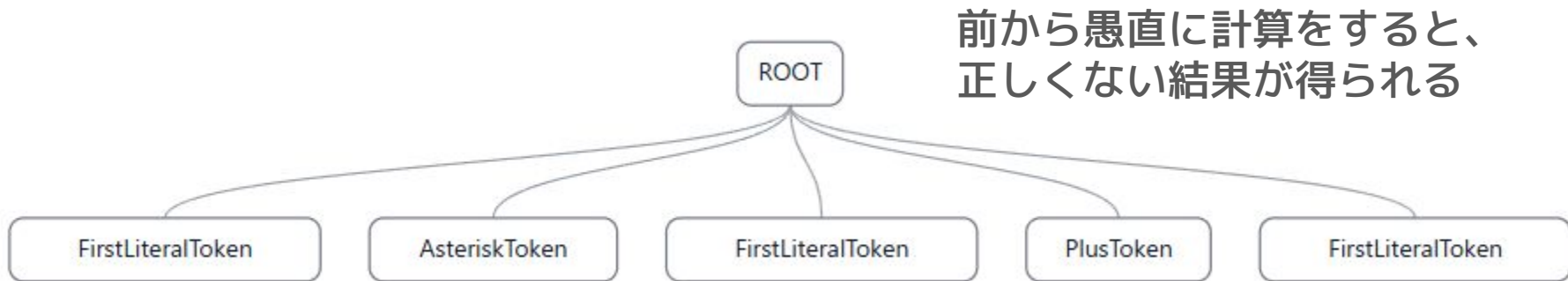
文脈自由文法（終端記号 / 非終端記号）

（算術式の例を含め）文脈自由文法の構文規則の定義において
非終端記号を使わずに、**式の優先順位は解析できない**

このようなトークン列を受理したいとき

3 FirstLiteralToken	* AsteriskToken	4 FirstLiteralToken	+ PlusToken	5 FirstLiteralToken
------------------------	--------------------	------------------------	----------------	------------------------

この構文木に式の優先順位の情報がない





構文解析

```
if (a){ . . . }  
else if (b){  
    if (b1){ . . . }  
    else  
    (b2){ . . . }  
}  
else if (c){ . . . }  
else { . . . }
```

こんなif文

正しく解析できる？

「同じ場所のtokenを何度も
読まない」という制約があったら？



構文解析

下向き構文解析
(トップダウン構文解析)

上向き構文解析
(ボトムアップ構文解析)



構文解析

下向き構文解析
(トップダウン構文解析)

上向き構文解析
(ボトムアップ構文解析)

木構造を上から作っていく



構文解析

下向き構文解析
(トップダウン構文解析)

上向き構文解析
(ボトムアップ構文解析)

木構造を **下から** 作っていく



構文解析

下向き構文解析
(トップダウン構文解析)

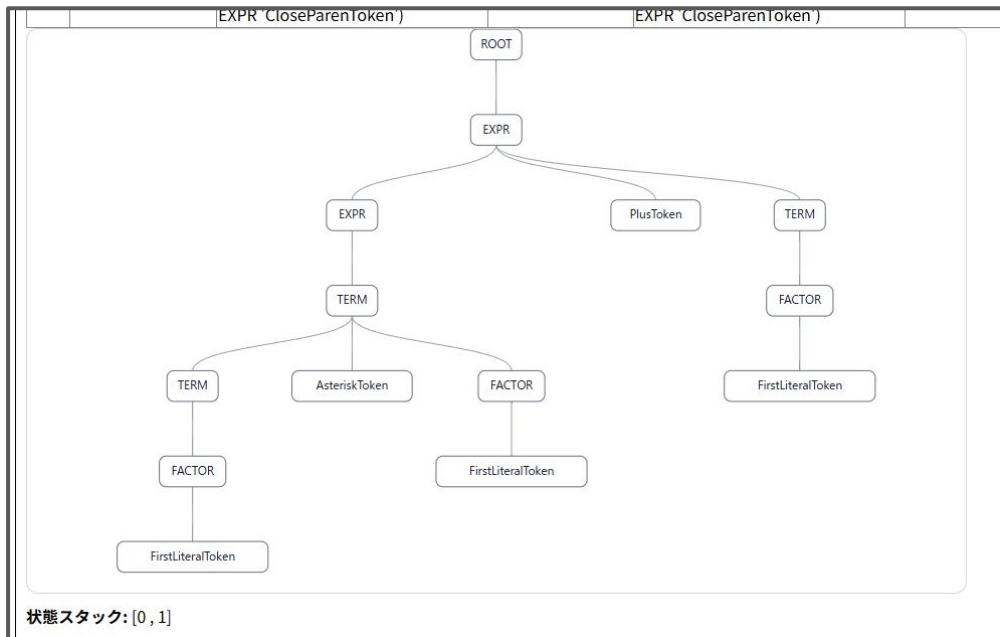
再帰下降
LL(k) LL(1)など

上向き構文解析
(ボトムアップ構文解析)

LR(k) LR(0)やLR(1)など
SLR LALR



構文解析



上向き構文解析
(ボトムアップ構文解析)

LR(0) / LR(1)



構文解析 LR法

LR(0)

アイテム集合

I0

LR項

(1行1行)

$S \rightarrow \bullet \text{EXPR}$

$\text{EXPR} \rightarrow \bullet \text{EXPR} \text{'PlusToken' TERM}$

$\text{EXPR} \rightarrow \bullet \text{EXPR} \text{'MinusToken' TERM}$

$\text{EXPR} \rightarrow \bullet \text{TERM}$

$\text{TERM} \rightarrow \bullet \text{TERM} \text{'AsteriskToken' FACTOR}$

$\text{TERM} \rightarrow \bullet \text{FACTOR}$

$\text{FACTOR} \rightarrow \bullet \text{'OpenParenToken' EXPR 'CloseParenToken'}$

$\text{FACTOR} \rightarrow \bullet \text{'MinusToken' FACTOR}$

$\text{FACTOR} \rightarrow \bullet \text{'FirstLiteralToken'}$



構文解析 LR法

LR(1)

先読み記号

I0

$S \rightarrow \bullet \text{EXPR} [\$]$

$\text{EXPR} \rightarrow \bullet \text{EXPR} \text{'PlusToken'} \text{TERM} [\$]$

$\text{EXPR} \rightarrow \bullet \text{EXPR} \text{'MinusToken'} \text{TERM} [\$]$

$\text{EXPR} \rightarrow \bullet \text{TERM} [\$]$

$\text{EXPR} \rightarrow \bullet \text{EXPR} \text{'PlusToken'} \text{TERM} [\text{PlusToken}]$

$\text{EXPR} \rightarrow \bullet \text{EXPR} \text{'MinusToken'} \text{TERM} [\text{PlusToken}]$

$\text{EXPR} \rightarrow \bullet \text{TERM} [\text{PlusToken}]$

$\text{EXPR} \rightarrow \bullet \text{EXPR} \text{'PlusToken'} \text{TERM} [\text{MinusToken}]$

$\text{EXPR} \rightarrow \bullet \text{EXPR} \text{'MinusToken'} \text{TERM} [\text{MinusToken}]$

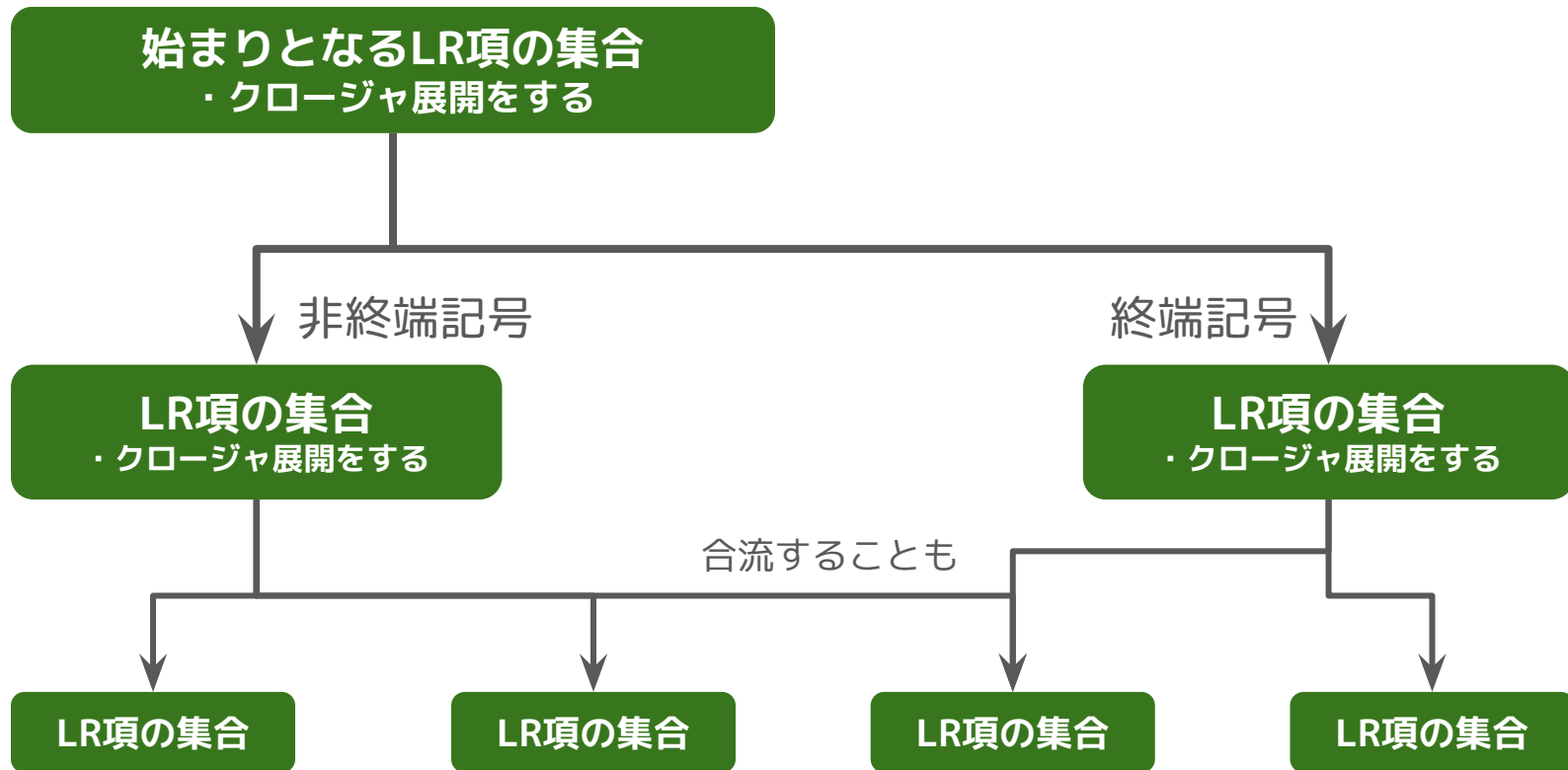
$\text{EXPR} \rightarrow \bullet \text{TERM} [\text{MinusToken}]$

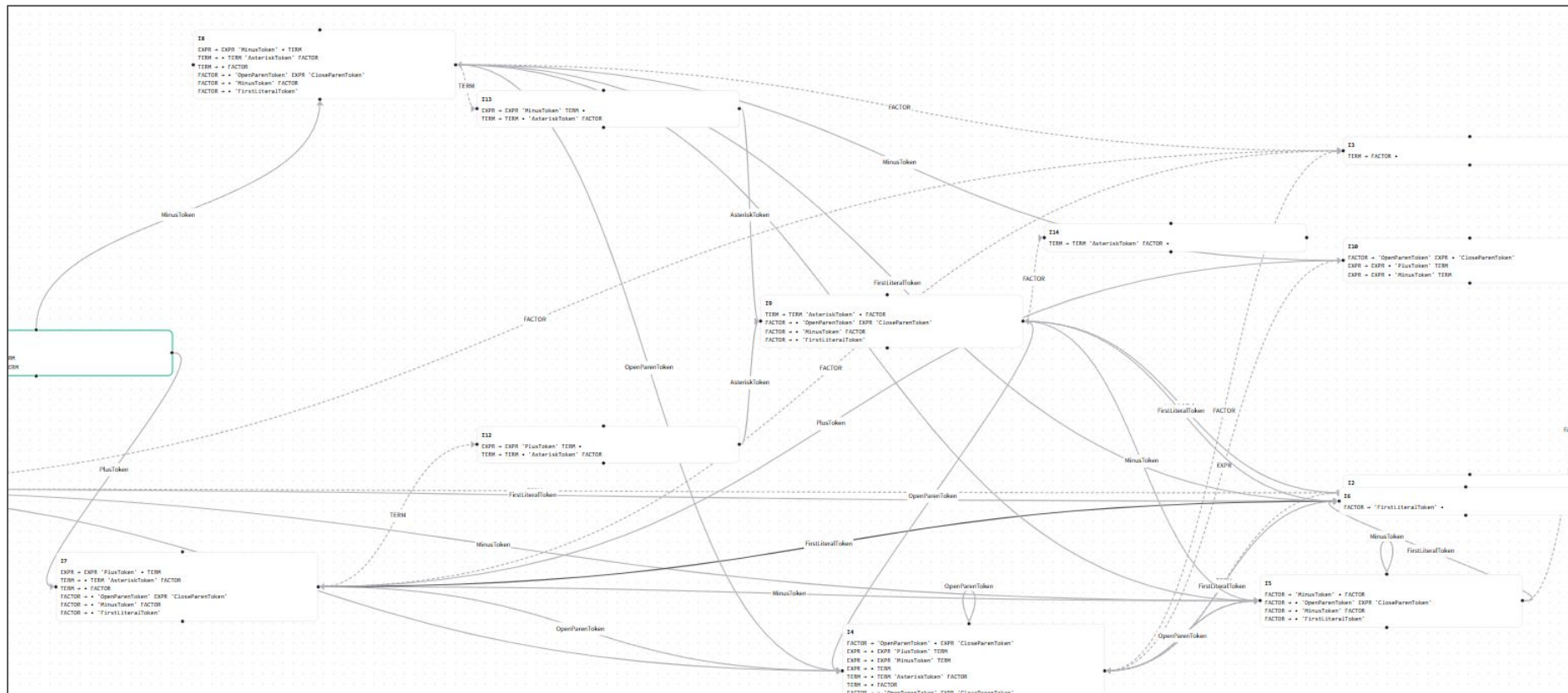
$\text{TERM} \rightarrow \bullet \text{TERM} \text{'AsteriskToken'} \text{FACTOR} [\$]$

$\text{TERM} \rightarrow \bullet \text{FACTOR} [\$]$



構文解析 LR法







構文解析 LR法

LR項

$$X \rightarrow \alpha \cdot Y \beta$$

マーカー(dot)

直感的には、「構文解析の途中経過位置」を表す

この場合、以下の直前の状態を表す

- ・ α までの解析を終了している
- ・ 次に $Y \beta$ の解析を始める



構文解析 LR法

LR項

$$X \rightarrow \alpha \cdot Y \beta$$

Y βの解析を行うためには、Yの解析が必要（部分問題として）
もし文法が $Y \rightarrow \gamma$ を含むなら γ の解析も必要

$X \rightarrow \alpha \cdot Y \beta$ があらわす構文解析の状態は $Y \rightarrow \cdot \gamma$ を含んでる
この関係を ↓を用いて

$[X \rightarrow \alpha \cdot Y \beta] \downarrow [Y \rightarrow \cdot \gamma]$ と表す
($[X \rightarrow \alpha \cdot Y \beta]$ が $[Y \rightarrow \cdot \gamma]$ を呼び出している)



LR(0)

$$\text{closure}(I) = I \cup \{i' \mid i \in \text{closure}(I), i \downarrow i'\}$$

初期集合 I に含まれるアイテムから始める
それらが導入するすべてのアイテムを
(さらにそこから導入できるものも含めて ≡再帰的に)
加えた集合



構文解析 LR法 クロージャ展開

LR(0)

$$\text{closure}(I) = I \cup \{i' \mid i \in \text{closure}(I), i \downarrow i'\}$$

1) LR項の集合 i' を I で初期化する

2) 以下の処理を i' が増えなくなるまで繰り返す

- a) i' に含まれるLR項の中で非終端記号の前にマーカー(dot)がついている形のものを $[X \rightarrow \alpha \cdot Y \beta]$ とするとき以下を行う
 - i. 文法に含まれる $Y \rightarrow \gamma$ の形の各構文規則についてLR項 $[Y \rightarrow \cdot \gamma]$ を i' に追加する

これで求まった I' を $\text{closure}(I)$ とする



構文解析 LR法 クロージャ展開

足し算の受理

2 + 3や、2 + 3 + 4 など...

LR(0)

$S \rightarrow \text{EXPR}$

$\text{EXPR} \rightarrow \text{EXPR PlusToken FirstLiteralToken}$
 $\quad \quad \quad | \text{FirstLiteralToken}$

$S \rightarrow \cdot \text{EXPR}$



$S \rightarrow \cdot \text{EXPR}$

$\text{EXPR} \rightarrow \cdot \text{EXPR PlusToken FirstLiteralToken}$

$\text{EXPR} \rightarrow \cdot \text{FirstLiteralToken}$

これって実は $X \rightarrow \alpha \cdot Y \beta$

LR項 $Y \rightarrow \cdot \gamma$ を l' に追加する必要がある

これをできる限り続ける (追加したものの含め)



構文解析 LR法 クロージャ展開

LR(0)

$$\text{closure}(I) = I \cup \{i' \mid i \in \text{closure}(I), i \downarrow i'\}$$

いかにも再帰っぽい感じ
気合で再帰なしで実装可能
(キューを用いた幅優先探索的手法)



構文解析 LR法 クロージャ展開

足し算の受理

2 + 3や、2 + 3 + 4 など...

LR(1)

$S \rightarrow \text{EXPR}$
 $\text{EXPR} \rightarrow \text{EXPR PlusToken FirstLiteralToken}$
 $\quad \mid \text{FirstLiteralToken}$

$S \rightarrow \cdot \text{EXPR} [\$]$

LR(1)の場合はさらに「先読み記号」が追加
その分 LR項が増える
(増えすぎるので、LALRというアルゴリズムがある)



$S \rightarrow \cdot \text{EXPR} [\$]$
 $\text{EXPR} \rightarrow \cdot \text{EXPR PlusToken FirstLiteralToken} [\$]$
 $\text{EXPR} \rightarrow \cdot \text{FirstLiteralToken} [\$]$
 $\text{EXPR} \rightarrow \cdot \text{EXPR PlusToken FirstLiteralToken} [\text{PlusToken}]$
 $\text{EXPR} \rightarrow \cdot \text{FirstLiteralToken} [\text{PlusToken}]$



LR(1)

T : 終端記号の有限集合

N : 非終端記号の有限集合

先読み記号の導出にはFirst集合が必要

$$\text{First}(\alpha) = \{ t \in T \mid \alpha \Rightarrow^* t \beta, \beta \in (T \cup N)^* \}$$

▲ $\text{First}(\alpha)$ は α から導出された列の先頭に現れることができる
終端記号の集合 なお、 $\alpha \in (T \cup N)^*$

α に対して、構文規則を何度か適用してすべて終端記号にした場合、先頭に出てくる可能性がある終端記号を集めた集合

$\Rightarrow^*$ 「何度かの変形で導出可能」



LR(1)

T : 終端記号の有限集合

N : 非終端記号の有限集合

先読み記号の導出にはFirst集合が必要

$$\text{First}(\alpha) = \{ t \in T \mid \alpha \Rightarrow^* t \beta, \beta \in (T \cup N)^* \}$$

▲ $\text{First}(\alpha)$ は α から導出された列の先頭に現れることができる
終端記号の集合 なお、 $\alpha \in (T \cup N)^*$

$X \rightarrow \alpha \cdot Y \beta [t]$

$Y \rightarrow \cdot \gamma [t']$

t' はどうやって計算？

β が空列になりうる時 (ϵ が出現するとき)

$\text{First}(\beta) \cup \{t\}$

そうではないとき $\text{First}(\beta)$



構文解析 LL法などの下向き構文解析（余談）

First集合の実用的な導出

空列 ϵ なら

$$\text{First}(\epsilon) = \emptyset \quad (\text{空集合})$$

先頭が終端記号であるような記号列 t_α なら

$$\text{First}(t_\alpha) = \{t\}$$

先頭が非終端記号であるような記号列 X_β で、

X が空列 ϵ に成りうるなら

$$\text{First}(X_\beta) = \text{First}(X) \cup \text{First}(\beta)$$

それ以外なら

$$\text{First}(X_\beta) = \text{First}(X)$$



構文解析 LL法などの下向き構文解析（余談）

Director集合

▲ Director集合：Xに適用すべき規則を指し示す終端記号の集合

$$\text{Director}(X \rightarrow \alpha) = \{t \in T \mid S \Rightarrow^* uX\beta \Rightarrow u\alpha\beta \Rightarrow^* ut\gamma, \\ u \in T^*, \quad \beta, \gamma \in (T \cup N)^*\}$$

- 1) 開始記号(S)から文形式 $uX\beta$ を導出
- 2) その文形式の中のXに $X \rightarrow \alpha$ を適用し、 $u\alpha\beta$ を導出
- 3) さらに最左導出（左から優先的に非終端記号を変形）を続ける

その結果 $ut\gamma$ が導出されるとき
そのような終端記号tをすべて集めた集合のこと



構文解析 LL法などの下向き構文解析（余談）

Director集合の実用的な導出

α が空列 ϵ に成りうるなら

$$\text{Director}(X \rightarrow \alpha) = \text{First}(\alpha) \cup \text{Follow}(X)$$

それ以外なら

$$\text{Director}(X \rightarrow \alpha) = \text{First}(\alpha)$$



構文解析 LL法などの下向き構文解析（余談）

Follow集合

上向き構文解析のSLRでも必要

$$\text{Follow}(X) = \{t \in \text{First}(\beta) \mid Z \Rightarrow^* u X \beta, u \in T^*, \beta \in (T \cup N)^*\}$$

- ▲ Follow(X) は、非終端記号Xの「後」に続いて現れる終端記号の集合
Director集合の導出に必要



構文解析 LL法などの下向き構文解析（余談）

Follow集合の実用的な導出

右辺にXが現れるような規則 $Y \rightarrow \alpha X \beta$ があるとして

$$\text{Follow}(X) = \bigcup_{i=1}^k \text{Follow}(\beta_i, Y_i)$$

1) 直後の文字が空列にならない場合
ただし、First集合に ϵ があるときは、 ϵ は除外すること

$$\text{Follow}(X) += \text{Fisrt}(\beta)$$

2) 直後の文字が空列になりうる場合

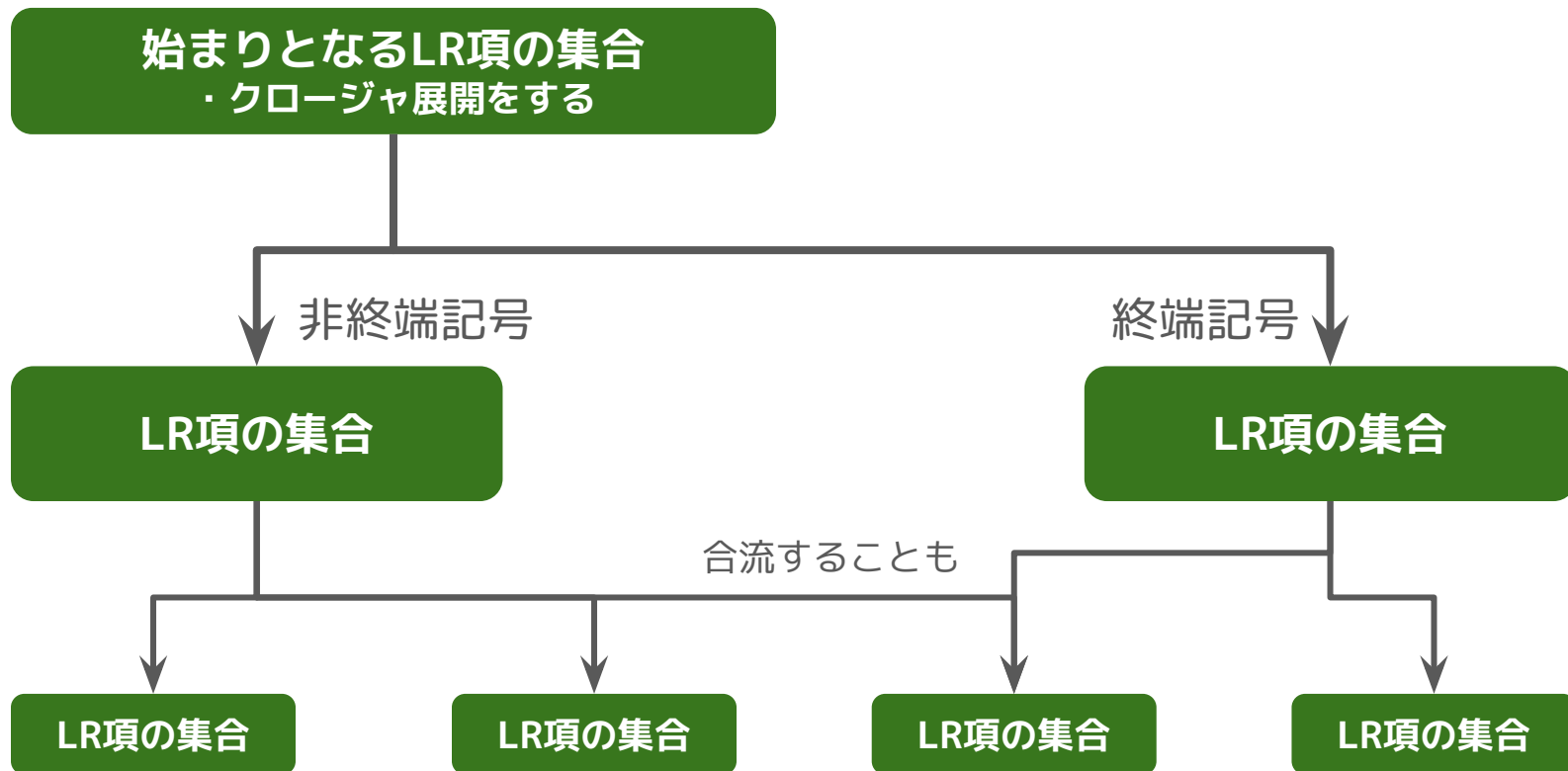
$$\text{Follow}(X) += \text{Follow}(Y)$$

なお、開始記号 S の直後は必ず文字列の終端 $\$$ $\text{Follow}(S) += \{\$ \}$

また、右再帰は無視してよい（間接的な右再帰の話はめんどいのでしません・・・）



構文解析 LR法





構文解析 LR法

LR項の集合

非終端記号

終端記号

LR項の集合

LR項の集合

I0

S → • EXPR [\$]

• EXPR → • EXPR 'PlusToken' 'FirstLiteralToken' [\$]

EXPR → • 'FirstLiteralToken' [\$]

EXPR → • EXPR 'PlusToken' 'FirstLiteralToken' [PlusToken]

EXPR → • 'FirstLiteralToken' [PlusToken]

FirstLiteralToken

EXPR

マーカーの次にある非終端記号と終端記号を用いて次のLR項の集合を作る



構文解析 LR法 クロージャ展開

```
9 namespace DFAParse
10 void ClosureExpansion::recursionNodeClosureExpansion(LRItemStruct &lr_item, string search_key, int LR_formula_expansion_vector_index, BNFParse::vD
11     for (int k = 0; k < dfexp.size(); k++)
12     {
13         if (!HasKeyMap(getMapKeyString(already_explored_formula_expansion), dfexp[k].formula_expansion_label))
14         {
15             int la_size = lr_item.LR_formula_map[token.token_str].LR_formula_expansion_vector[n_index].look_ahead.size();
16             for (int t = 0; t < la_size; t++)
17             {
18                 BNFParse::vDeploymentTokenStruct new_first_set = getLatterFirstSet(lr_item.LR_formula_map[token.token_str].LR_formula_expansion_ve
19                 recursionNodeClosureExpansion(lr_item, token.token_str, n_index, new_first_set);
20             }
21         }
22     }
23 else
24 { // 先読み記号だけ追加する
25     int formula_expansion_index = already_explored_formula_expansion[dfexp[k].formula_expansion_label];
26     BNFParse::vDeploymentTokenStruct *current_look_ahead_p = &(lr_item.LR_formula_map[token.token_str].LR_formula_expansion_vector[formula
27
28     for (int n = 0; n < first_set.size(); n++)
29     {
30         bool flag = true; // 既に登録されているか
31         for (int s = 0; s < current_look_ahead_p->size(); s++)
32         {
33             if (current_look_ahead_p->at(s).token_str == first_set[n].token_str)
34             {
35                 flag = false;
36                 break;
37             }
38         }
39         if (flag)
40         {
41             current_look_ahead_p->push_back(first_set[n]);
42
43             int dot = lr_item.LR_formula_map[token.token_str].LR_formula_expansion_vector[formula_expansion_index].dot;
44             BNFParse::vDeploymentTokenStruct new_first_set = getLatterFirstSet(lr_item.LR_formula_map[token.token_str].LR_formula_expansio
45             recursionNodeClosureExpansion(lr_item, token.token_str, formula_expansion_index, new_first_set);
46         }
47     }
48 }
49 }
```



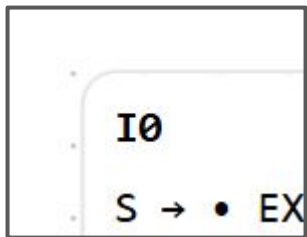
LR構文解析（構文解析表）

State	Actions							Gotos		
	OpenParenToken	MinusToken	FirstLiteralToken	PlusToken	\$	AsteriskToken	CloseParenToken	EXPR	TERM	FACTOR
0	S4	S5	S6					1	2	3
1		S8		S7	ACC					
2		R(EXPR -> TERM)		R(EXPR -> TERM)	R(EXPR -> TERM)	S9				
3		R(TERM -> FACTOR)		R(TERM -> FACTOR)	R(TERM -> FACTOR)	R(TERM -> FACTOR)				
4	S13	S14	S15					10	11	12
5	S4	S5	S6							16
6		R(FACTOR -> 'FirstLiteralToken')		R(FACTOR -> 'FirstLiteralToken')	R(FACTOR -> 'FirstLiteralToken')	R(FACTOR -> 'FirstLiteralToken')				
7	S4	S5	S6						17	3
8	S4	S5	S6						18	3
9	S4	S5	S6							19
10		S22		S21			S20			
11		R(EXPR -> TERM)		R(EXPR -> TERM)		S23	R(EXPR -> TERM)			
12		R(TERM -> FACTOR)		R(TERM -> FACTOR)		R(TERM -> FACTOR)	R(TERM -> FACTOR)			
13	S13	S14	S15					24	11	12
14	S13	S14	S15							25
15		R(FACTOR -> 'FirstLiteralToken')		R(FACTOR -> 'FirstLiteralToken')		R(FACTOR -> 'FirstLiteralToken')	R(FACTOR -> 'FirstLiteralToken')			
16		R(FACTOR -> 'MinusToken' FACTOR)		R(FACTOR -> 'MinusToken' FACTOR)	R(FACTOR -> 'MinusToken' FACTOR)	R(FACTOR -> 'MinusToken' FACTOR)				
17		R(EXPR -> EXPR 'PlusToken' TERM)		R(EXPR -> EXPR 'PlusToken' TERM)	R(EXPR -> EXPR 'PlusToken' TERM)	S9				
18		R(EXPR -> EXPR 'MinusToken' TERM)		R(EXPR -> EXPR 'MinusToken' TERM)	R(EXPR -> EXPR 'MinusToken' TERM)	S9				
19		R(TERM -> TERM 'AsteriskToken' FACTOR)		R(TERM -> TERM 'AsteriskToken' FACTOR)	R(TERM -> TERM 'AsteriskToken' FACTOR)	R(TERM -> TERM 'AsteriskToken' FACTOR)				
20		R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')		R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')				
21	S13	S14	S15						26	12
22	S13	S14	S15						27	12
23	S13	S14	S15							28
24		S22		S21			S29			
25		R(FACTOR -> 'MinusToken' FACTOR)		R(FACTOR -> 'MinusToken' FACTOR)		R(FACTOR -> 'MinusToken' FACTOR)	R(FACTOR -> 'MinusToken' FACTOR)			
26		R(EXPR -> EXPR 'PlusToken' TERM)		R(EXPR -> EXPR 'PlusToken' TERM)	S23		R(EXPR -> EXPR 'PlusToken' TERM)			
27		R(EXPR -> EXPR 'MinusToken' TERM)		R(EXPR -> EXPR 'MinusToken' TERM)	S23		R(EXPR -> EXPR 'MinusToken' TERM)			
28		R(TERM -> TERM 'AsteriskToken' FACTOR)		R(TERM -> TERM 'AsteriskToken' FACTOR)		R(TERM -> TERM 'AsteriskToken' FACTOR)	R(TERM -> TERM 'AsteriskToken' FACTOR)			
29		R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')		R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')		R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')			



LR構文解析（構文解析表）

縦のラベルは状態番号
(LR項の集合の番号)



State		Actions						Gotos		
	OpenParenToken	MinusToken	FirstLiteralToken	PlusToken	\$	AsteriskToken	CloseParenToken	EXPR	TERM	FACTOR
0	S4	S5	S6	S7	ACC			1	2	3
1		S8								
2		R(EXPR -> TERM)		R(EXPR -> TERM)	R(EXPR -> TERM)	S9				
3		R(TERM -> FACTOR)		R(TERM -> FACTOR)	R(TERM -> FACTOR)					
4	S13	S14	S15					10	11	12
5	S4	S5	S6							16
6		R(FACTOR -> 'FirstLiteralToken')		R(FACTOR -> 'FirstLiteralToken')	R(FACTOR -> 'FirstLiteralToken')	R(FACTOR -> 'FirstLiteralToken')				
7	S4	S5	S6					17	3	
8	S4	S5	S6					18	3	
9	S4	S5	S6							19
10		S22		S21			S20			
11		R(EXPR -> TERM)		R(EXPR -> TERM)		S23	R(EXPR -> TERM)			
12		R(TERM -> FACTOR)		R(TERM -> FACTOR)			R(TERM -> FACTOR)			
13	S13	S14	S15					24	11	12
14	S13	S14	S15							25
15		R(FACTOR -> 'FirstLiteralToken')		R(FACTOR -> 'FirstLiteralToken')		R(FACTOR -> 'FirstLiteralToken')	R(FACTOR -> 'FirstLiteralToken')			
16		R(FACTOR -> 'MinusToken' FACTOR)		R(FACTOR -> 'MinusToken' FACTOR)	R(FACTOR -> 'MinusToken' FACTOR)	R(FACTOR -> 'MinusToken' FACTOR)				
17		R(EXPR -> EXPR 'PlusToken' TERM)		R(EXPR -> EXPR 'PlusToken' TERM)	R(EXPR -> EXPR 'PlusToken' TERM)					
18		R(EXPR -> EXPR 'MinusToken' TERM)		R(EXPR -> EXPR 'MinusToken' TERM)	R(EXPR -> EXPR 'MinusToken' TERM)	S9				
19		R(TERM -> TERM 'AsteriskToken' FACTOR)		R(TERM -> TERM 'AsteriskToken' FACTOR)	R(TERM -> TERM 'AsteriskToken' FACTOR)	R(TERM -> TERM 'AsteriskToken' FACTOR)				
20		R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')		R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')				
21	S13	S14	S15					26	12	
22	S13	S14	S15					27	12	
23	S13	S14	S15							28
24		S22		S21			S29			
25		R(FACTOR -> 'MinusToken' FACTOR)		R(FACTOR -> 'MinusToken' FACTOR)		R(FACTOR -> 'MinusToken' FACTOR)	R(FACTOR -> 'MinusToken' FACTOR)			
26		R(EXPR -> EXPR 'PlusToken' TERM)		R(EXPR -> EXPR 'PlusToken' TERM)		S23	R(EXPR -> EXPR 'PlusToken' TERM)			
27		R(EXPR -> EXPR 'MinusToken' TERM)		R(EXPR -> EXPR 'MinusToken' TERM)			R(EXPR -> EXPR 'MinusToken' TERM)			
28		R(TERM -> TERM 'AsteriskToken' FACTOR)		R(TERM -> TERM 'AsteriskToken' FACTOR)		R(TERM -> TERM 'AsteriskToken' FACTOR)	R(TERM -> TERM 'AsteriskToken' FACTOR)			
29		R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')		R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')		R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')			

Action部(表) ラベルは終端記号

Goto部(表) ラベルは非終端記号



LR構文解析（構文解析表の要素）

Accept

構文解析が成功したことを表す

Reduce(構文規則)

- 1) その構文規則で還元 stackからその構文規則の長さだけpop
- 2) さらにstackのtopの値からgoto部を参照 その要素をstackにpush

Shift(遷移先の状態番号)

- 1) トークン列の現在のトークンと現在の状態番号から shift 表を参照
- 2) その表の要素をスタックにプッシュする



LR構文解析（構文解析表の作成）

LR(0)

Accept

$S \rightarrow X \cdot$ があるLR項の集合 がある行に\$をkeyとして、そこに書き込む

Reduce(構文規則)

アイテム集合*i*において

還元項 $X \rightarrow \alpha \cdot$ があるとき、*i*行のすべての終端記号にReduce操作を加える

Shift(遷移先の状態番号)

マーカの後ろに要素がある場合、その遷移先の状態番号を記載

$X \rightarrow \alpha \cdot Y \beta$ があるならば、接続している $X \rightarrow \alpha Y \cdot \beta$ がある遷移の番号を記載



LR構文解析（構文解析表の作成）

LR(1)

Accept

$S \rightarrow X \cdot$ があるLR項の集合 がある行に\$をkeyとして、そこに書き込む

Reduce(構文規則)

還元項 $X \rightarrow \alpha \cdot [t]$ があるとき、 t 列の終端記号にReduce操作を加える

Shift(遷移先の状態番号)

マーカーの後ろに要素がある場合、その遷移先の状態番号を記載

$X \rightarrow \alpha \cdot Y \beta$ があるならば、接続している $X \rightarrow \alpha Y \cdot \beta$ がある遷移の番号を記載



LR法 構文解析の証明

まず、文脈自由文法において、
すべての非終端記号 A は、 S (開始記号)から到達可能で、
 $w \in T^*$ に対して $A \Rightarrow^* w$ としてみる (一般性は失わないので大丈夫)

$S \rightarrow a$
 $A \rightarrow b$

こういう文法があったとしても
 $A \rightarrow b$ は使われないので、消してしまっても
生成される言語は同じ



LR構文解析（ボトムアップ構文解析）

LR(0)法とその弱さ

State	Actions							Gotos		
	OpenParenToken	MinusToken	FirstLiteralToken	\$	PlusToken	AsteriskToken	CloseParenToken	EXPR	TERM	FACTOR
0	S4	S5	S6					1	2	3
1		S8		ACC	S7					
2	R(EXPR → TERM)	R(EXPR → TERM)	R(EXPR → TERM)	R(EXPR → TERM)	R(EXPR → TERM)	R(EXPR → TERM)	R(EXPR → TERM)			
3	R(TERM → FACTOR)	R(TERM → FACTOR)	R(TERM → FACTOR)	R(TERM → FACTOR)	R(TERM → FACTOR)	R(TERM → FACTOR)	R(TERM → FACTOR)			
4	S4	S5	S6					1	2	3
5	S4	S5	S6							11
6	R(FACTOR → 'FirstLiteralToken')	R(FACTOR → 'FirstLiteralToken')	R(FACTOR → 'FirstLiteralToken')	R(FACTOR → 'FirstLiteralToken')	R(FACTOR → 'FirstLiteralToken')	R(FACTOR → 'FirstLiteralToken')	R(FACTOR → 'FirstLiteralToken')			
7	S4	S5	S6					2	3	
8	S4	S5	S6					2	3	
9	S4	S5	S6							14
10		S8			S7		S15			
11	R(FACTOR → 'MinusToken' FACTOR)	R(FACTOR → 'MinusToken' FACTOR)	R(FACTOR → 'MinusToken' FACTOR)	R(FACTOR → 'MinusToken' FACTOR)	R(FACTOR → 'MinusToken' FACTOR)	R(FACTOR → 'MinusToken' FACTOR)	R(FACTOR → 'MinusToken' FACTOR)			
12	R(EXPR → EXPR 'PlusToken' TERM)	R(EXPR → EXPR 'PlusToken' TERM)	R(EXPR → EXPR 'PlusToken' TERM)	R(EXPR → EXPR 'PlusToken' TERM)	R(EXPR → EXPR 'PlusToken' TERM)	R(EXPR → EXPR 'PlusToken' TERM)	R(EXPR → EXPR 'PlusToken' TERM)			
13	R(EXPR → EXPR 'MinusToken' TERM)	R(EXPR → EXPR 'MinusToken' TERM)	R(EXPR → EXPR 'MinusToken' TERM)	R(EXPR → EXPR 'MinusToken' TERM)	R(EXPR → EXPR 'MinusToken' TERM)	R(EXPR → EXPR 'MinusToken' TERM)	R(EXPR → EXPR 'MinusToken' TERM)			
14	R(TERM → TERM 'AsteriskToken' FACTOR)	R(TERM → TERM 'AsteriskToken' FACTOR)	R(TERM → TERM 'AsteriskToken' FACTOR)	R(TERM → TERM 'AsteriskToken' FACTOR)	R(TERM → TERM 'AsteriskToken' FACTOR)	R(TERM → TERM 'AsteriskToken' FACTOR)	R(TERM → TERM 'AsteriskToken' FACTOR)			
15	R(FACTOR → 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR → 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR → 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR → 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR → 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR → 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR → 'OpenParenToken' EXPR 'CloseParenToken')			

LR(0)の構文解析表作成においては アイテム集合*i*において

還元項 $[X \rightarrow \alpha \cdot]$ があるとき、*i*行の**すべての終端記号**にReduce操作を加える



LR構文解析（ボトムアップ構文解析）

LR(0)法とその弱さ

LR(0)の構文解析表作成においては アイテム集合*i*において
還元項 $[X \rightarrow \alpha \cdot]$ があるとき、*i*行の**すべての終端記号**にReduce操作を加える

State				Actions
	OpenParenToken	MinusToken	FirstLiteralToken	\$
0	S4	S5	S6	
1		S5		ACC
2	R(EXPR -> TERM)	R(EXPR -> TERM)	R(EXPR -> TERM)	R(EXPR -> TERM)
3	R(TERM -> FACTOR)	R(TERM -> FACTOR)	R(TERM -> FACTOR)	R(TERM -> FACTOR)
4	S4	S5	S6	
5	S4	S5	S6	
6	R(FACTOR -> 'FirstLiteralToken')	R(FACTOR -> 'FirstLiteralToken')	R(FACTOR -> 'FirstLiteralToken')	R(FACTOR -> 'FirstLiteralToken')
7	S4	S5	S6	
8	S4	S5	S6	
9	S4	S5	S6	
10		S5		
11	R(FACTOR -> 'MinusToken' FACTOR)	R(FACTOR -> 'MinusToken' FACTOR)	R(FACTOR -> 'MinusToken' FACTOR)	R(FACTOR -> 'MinusToken' FACTOR)
12	R(EXPR -> EXPR 'PlusToken' TERM)	R(EXPR -> EXPR 'PlusToken' TERM)	R(EXPR -> EXPR 'PlusToken' TERM)	R(EXPR -> EXPR 'PlusToken' TERM)
13	R(EXPR -> EXPR 'MinusToken' TERM)	R(EXPR -> EXPR 'MinusToken' TERM)	R(EXPR -> EXPR 'MinusToken' TERM)	R(EXPR -> EXPR 'MinusToken' TERM)
14	R(TERM -> TERM 'AsteriskToken' FACTOR)	R(TERM -> TERM 'AsteriskToken' FACTOR)	R(TERM -> TERM 'AsteriskToken' FACTOR)	R(TERM -> TERM 'AsteriskToken' FACTO
15	R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')



LR構文解析（ボトムアップ構文解析）

LR(0)法とその弱さ

LR(0)の構文解析表作成においては アイテム集合*i*において
還元項 $[X \rightarrow \alpha \cdot]$ があるとき、*i*行の**すべての終端記号**にReduce操作を加える

	PlusToken	AsteriskToken	CloseParenToken	
				E
				1
S7				
R(EXPR -> TERM)		C(R(EXPR -> TERM), S9)	R(EXPR -> TERM)	
R(TERM -> FACTOR)		R(TERM -> FACTOR)	R(TERM -> FACTOR)	
				1
R(FACTOR -> 'FirstLiteralToken')		R(FACTOR -> 'FirstLiteralToken')	R(FACTOR -> 'FirstLiteralToken')	
S7			S15	
R(FACTOR -> 'MinusToken' FACTOR)		R(FACTOR -> 'MinusToken' FACTOR)	R(FACTOR -> 'MinusToken' FACTOR)	
R(EXPR -> EXPR 'PlusToken' TERM)		C(R(EXPR -> EXPR 'PlusToken' TERM), S9)	R(EXPR -> EXPR 'PlusToken' TERM)	
R(EXPR -> EXPR 'MinusToken' TERM)		C(R(EXPR -> EXPR 'MinusToken' TERM), S9)	R(EXPR -> EXPR 'MinusToken' TERM)	
R(TERM -> TERM 'AsteriskToken' FACTOR)		R(TERM -> TERM 'AsteriskToken' FACTOR)	R(TERM -> TERM 'AsteriskToken' FACTOR)	
R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')		R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')	R(FACTOR -> 'OpenParenToken' EXPR 'CloseParenToken')	

すぐに衝突してしまう



LR構文解析（ボトムアップ構文解析）

愚直にクロージャ計算すると、LR(1)でも衝突が起きる例
（空文字関連は起きやすい）

$S \rightarrow \text{IF ELSEIF ELSE}$

$\text{IF} \rightarrow \text{IfKeyword Identifier}$

$\text{ELSEIF} \rightarrow \text{ElseKeyword IfKeyword Identifier} \mid \epsilon$

$\text{ELSE} \rightarrow \text{ElseKeyword Identifier}$

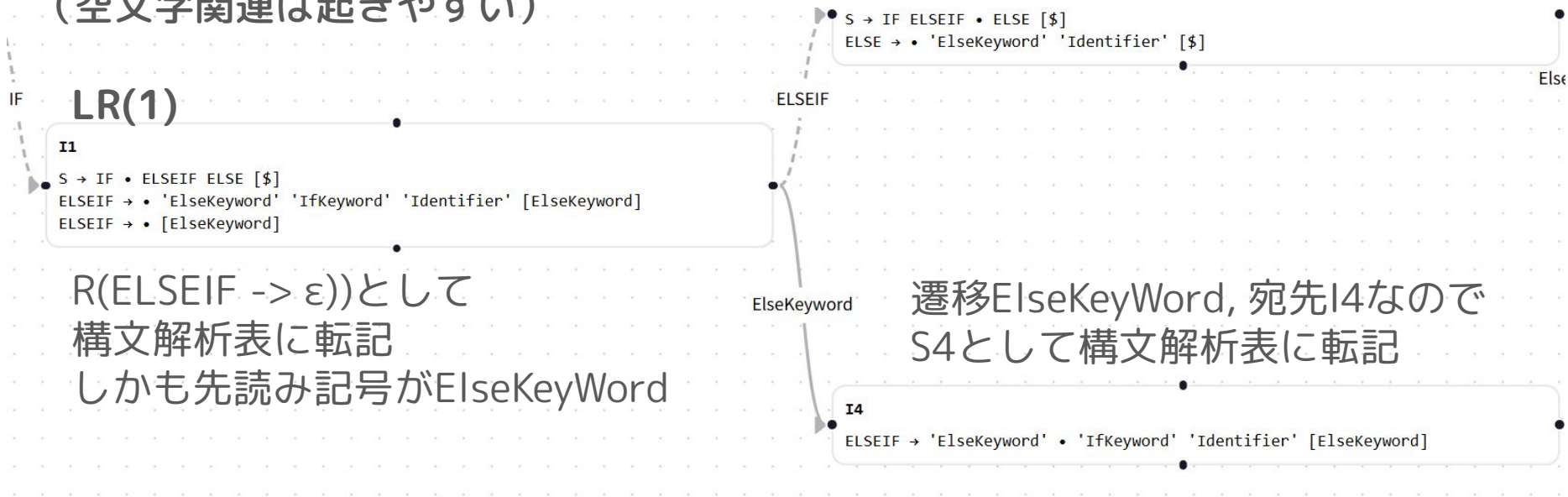
* わかりやすいように、極端な例にしています

* ほぼ実用的ではない構文定義



LR構文解析（ボトムアップ構文解析）

愚直にクロージャ計算すると、LR(1)でも衝突が起きる例
（空文字関連は起きやすい）





LR構文解析（ボトムアップ構文解析）

LR(1)

State	Actions				Gotos		
	IfKeyword	ElseKeyword	Identifier	\$	IF	ELSEIF	ELSE
0	S2				1		
1		C(S4, R(ELSEIF ->))			3		
2			S5				
3		S7					6
4	S8						
5		R(IF -> 'IfKeyword' 'Identifier')					
6				ACC			
7			S9				
8			S10				
9				R(ELSE -> 'ElseKeyword' 'Identifier')			
10		R(ELSEIF -> 'ElseKeyword' 'IfKeyword' 'Identifier')					

コンフリクトが発生しているセルは赤色で表示されます。

コンフリクトが発生しています！

状態 1:

- トークン 'ElseKeyword' に対するアクションがコンフリクトしています:
 - シフト to 状態 4
 - リダクション by ELSEIF ->

コンフリクトを解消するには、生成規則の見直しや優先順位の設定を検討してください。



LR構文解析（ボトムアップ構文解析）

あいまいさの解決で、
shift-reduce衝突はshiftを優先するという手段

shift と reduce を選択する方法はありません。これは、shift-reduce 衝突と呼ばれています。また、正当な2つの還元を選択肢をパーサーが持って shift-reduce 衝突や reduce-reduce 衝突が生じた場合でも、yacc はパーサーを生成します。yacc は、選択肢がある場所で有効なステップの1つを選択と呼ばれます。

yacc は、デフォルトの2つのあいまいさ解決規則を呼び出します。

1. shift-reduce 衝突の場合は、デフォルトは shift になります。
2. reduce-reduce 衝突の場合は、デフォルトは、読み込まれた yacc 仕様中で先に定義された文法規則による reduce になります。

規則1は、選択肢があるところでは還元はシフトの方に従うことを意味しています。規則2では、ユーザーがこの状況において行えるパー
衝突は、入力や論理に誤りがあるとき、または yacc では構築できない複雑なパーサーが(矛盾のない) 構文規則で必要になったときに発生し
則内のアクションを使用すると衝突が発生することがあります。

これらの場合にあいまいさ解決規則を適用するのは不適切であり、パーサーも不正なものになります。そのため、yacc は上記の規則1と規
一般に、あいまいさ解決規則を適用して正しいパーサーを生成できるときには、構文規則を書き直して同じ入力を読み取るようにすること
衝突は重大なエラーとして扱われていました。

このような書き直しは、やや不自然である上に、低速のパーサーが生成されることになります。そのため、yacc は衝突が存在している場合

oracleのドキュメント

Documentation Home > プログラミングユーティリティ

第3章 yacc - コンパイラコンパイラ > あいまいさと衝突

<https://docs.oracle.com/cd/E19620-01/805-5827/6j5qfran2/index.html>



LR構文解析（ボトムアップ構文解析）

S -> IF ELSEIF ELSE

IF -> IfKeyword Identifier

ELSEIF -> ElseKeyword IfKeyword Identifier | ϵ

ELSE -> ElseKeyword Identifier

State	Actions				Gotos		
	IfKeyword	ElseKeyword	Identifier	\$	IF	ELSEIF	ELSE
0	S2				1		
1		S4			3		
2			S5				
3		S7				6	
4	S8						
5		R(IF -> 'IfKeyword' 'Identifier')					
6				ACC			
7			S9				
8			S10				
9				R(ELSE -> 'ElseKeyword' 'Identifier')			
10		R(ELSEIF -> 'ElseKeyword' 'IfKeyword' 'Identifier')					

入力

if

a

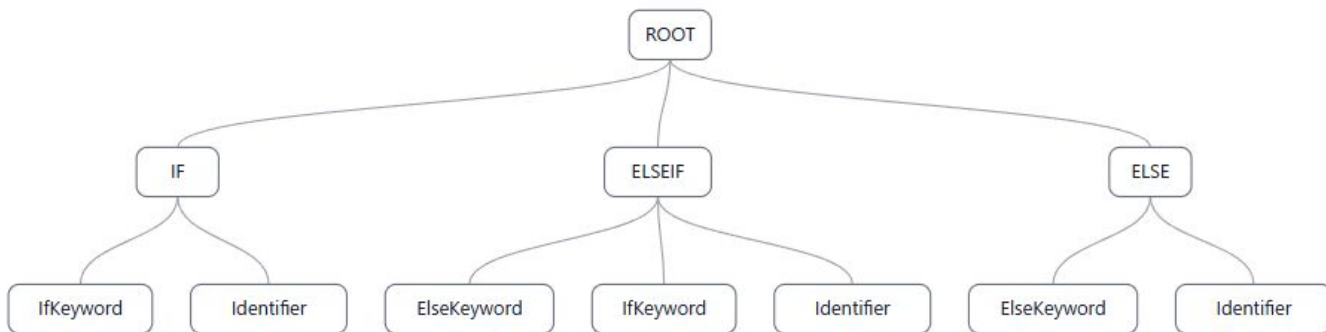
else

if

b

else

c





LR構文解析（ボトムアップ構文解析）

S -> IF ELSEIF ELSE

IF -> IfKeyword Identifier

ELSEIF -> ElseKeyword IfKeyword Identifier | ϵ

ELSE -> ElseKeyword Identifier

入力

if

a

else

c

State	Actions				Gotos		
	IfKeyword	ElseKeyword	Identifier	\$	IF	ELSEIF	ELSE
0	S2				1		
1		S4			3		
2			S5				
3		S7					6
4	S8						
5		R(IF -> 'IfKeyword' 'Identifier')					
6			ACC				
7			S9				
8			S10				
9				R(ELSE -> 'ElseKeyword' 'Identifier')			
10		R(ELSEIF -> 'ElseKeyword' 'IfKeyword' 'Identifier')					

State	Actions				Gotos		
	IfKeyword	ElseKeyword	Identifier	\$	IF	ELSEIF	ELSE
0	S2				1		
1		R(S4, R(ELSEIF ->))			3		
2			S5				
3		S7					6
4	S8						
5		R(IF -> 'IfKeyword' 'Identifier')					
6			ACC				
7			S9				
8			S10				
9				R(ELSE -> 'ElseKeyword' 'Identifier')			
10		R(ELSEIF -> 'ElseKeyword' 'IfKeyword' 'Identifier')					

shift-reduce衝突
握り潰し

R(ELSEIF -> ϵ))が消滅
構文解析できない



LR構文解析（ボトムアップ構文解析）

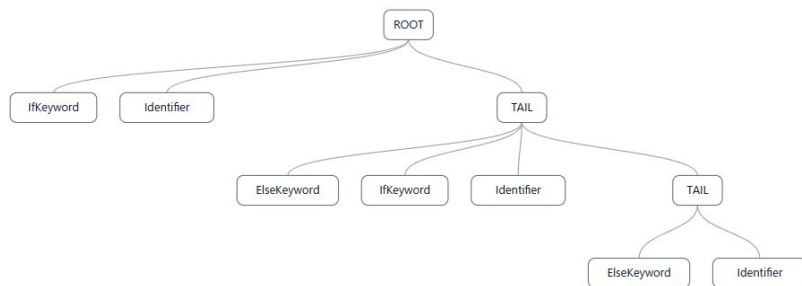
そもそも、そんな構文規則にしない

$S \rightarrow \text{IfKeyword Identifier TAIL}$

$\text{TAIL} \rightarrow \text{ElseKeyword IfKeyword Identifier TAIL} \mid \text{ElseKeyword Identifier} \mid \varepsilon$

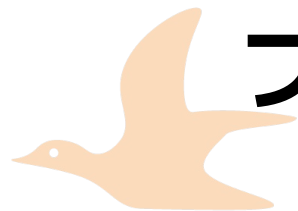
これなら、LR(0)には収まらないが、LR(1)には収まる

State	Actions				Gotos
	IfKeyword	Identifier	ElseKeyword	\$	TAIL
0	S1				
1		S2			
2			S4	R(TAIL ->)	3
3				ACC	
4	S5	S6			
5		S7			
6				R(TAIL -> 'ElseKeyword' 'Identifier')	
7			S4	R(TAIL ->)	8
8				R(TAIL -> 'ElseKeyword' 'IfKeyword' 'Identifier' TAIL)	



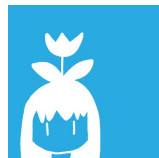
if-elseの順序関係は木構造の性質を用いて表現すべき

プログラミング言語処理系入門



プログラミング言語を作る理論

コンピューターは四則演算の順序をいかに認識するのか



X(Twitter) : ShioPy0101

主な参考文献

- Robery Nystrom 著 吉川邦夫 訳 ,CRAFTING INTERPRETERS インタプリタの作り方 -言語設計/開発の基本と 2 つの方式による実装-,株式会社インプレス,2023 年
- 大堀淳, LR 構文解析の原理, コンピュータソフトウェア, vol.31, no.1, pp. 1_30 - 1_42, 2014 年
- Andrew W Apple 著 神林 靖・滝本 宗宏 翻訳 ,最新コンパイラ構成技法(日本語訳) Modern Compiler Implementation in ML New Edition , 翔泳社 , 2009 年
- 山下義行 著 ,コンパイラ入門 -構文解析の原理と lex yacc C 言語による実装-,サイエンス社, 2008 年